

Competitive Programming Notebook

August 10, 2026

Contents

1	Miscellaneous	1			
1.1	Template	1			
1.2	fastIO	1			
1.3	grid	1			
1.4	random	1			
1.5	coordinateCompression	1			
1.6	grayCode	1			
1.7	customHashHashTable	1			
1.8	pragmas	2			
1.9	int128	2			
2	Data Structures (DS)	2			
2.1	Segment Tree	2			
2.1.1	SegmentTree	2			
2.1.2	LazySegmentTree	2			
2.1.3	PersistentSegmentTree	2			
2.1.4	PersistentSegTreeVector	3			
2.1.5	SegmentTreeBeats	3			
2.1.6	DynamicSegTree	4			
2.1.7	MergeSortTree	5			
2.1.8	IterativeSegTree	5			
2.1.9	SegTree2D	5			
2.2	Binary Indexed Tree (BIT)	5			
2.2.1	FenwickTree	5			
2.2.2	FenwickTree2D	5			
2.2.3	Offline2DFenwickTree	6			
2.3	Sparse Table	6			
2.3.1	SparseTable	6			
2.3.2	SparseTable2D	6			
2.3.3	SquareSparseTable	6			
2.4	Trie	6			
2.4.1	StringTrie	6			
2.4.2	BinaryTrie	7			
2.4.3	PersistentTrie	7			
2.5	Disjoint Set Union (DSU)	7			
2.5.1	DSU	7			
2.5.2	DSURollback	7			
2.6	Square Root Decomposition	8			
2.6.1	MoAlgorithm	8			
2.6.2	MoOnSubtrees	8			
2.6.3	MoOnPaths	8			
2.7	Balanced Binary Search Tree (BBST)	9			
2.7.1	Treap	9			
2.7.2	ImplicitTreap	10			
2.8	Policy Based Data Structures	11			
2.8.1	OrderedSet	11			
2.9	Monotonic Data Structures	11			
2.9.1	MonotonicStack	11			
2.9.2	TwoStackQueue	11			
2.9.3	MonotonicQueue2D	11			
3	Graph Theory	12			
3.1	Graph Traversal	12			
3.1.1	TopoSort	12			
3.2	Shortest Paths	12			
3.2.1	Dijkstra	12			
3.2.2	FloydWarshall	12			
3.2.3	BellmanFord	12			
3.3	Graph Connectivity	12			
3.3.1	Bridges	12			
3.3.2	CutPoints	12			
3.3.3	SCC	13			
3.4	Satisfiability	13			
3.4.1	2SAT	13			
3.5	Eulerian Path	13			
3.5.1	EulerianPath	13			
3.6	Flow and Min Cut	14			
			3.6.1	Dinic	14
			3.6.2	MCMF	14
			3.7	Matching	14
			3.7.1	HopcroftKarp	14
4	Number Theory	14			
4.1	Euclidean Algorithm	14			
4.1.1	ExtendedEuclid	14			
4.1.2	LinearDiophantine	15			
4.2	Primes and Divisors	15			
4.2.1	Sieve	15			
4.2.2	Factorization	15			
4.2.3	Divisors	15			
4.2.4	Sieve1e9	15			
4.2.5	SegmentedSieve	15			
4.3	Modular Arithmetic	15			
4.3.1	CRT	15			
4.3.2	FloorSum	16			
5	Combinatorics	16			
5.1	StarsAndBars	16			
5.2	BurnsideLemma	16			
6	Math	16			
6.1	Modular Arithmetic	16			
6.1.1	ModularArithmetic	16			
6.1.2	ModularIntClass	16			
6.2	Multiplicative Functions	17			
6.2.1	EulerTotientPhi	17			
6.2.2	MobiusFunction	17			
6.3	Matrices	17			
6.3.1	MatrixExponentiation	17			
6.3.2	FastMatrixPower	17			
6.4	Linear Algebra	17			
6.4.1	LinearXorBasis	17			
6.5	Convolution	18			
6.5.1	FFT	18			
6.5.2	NTT	18			
6.5.3	FWHT	18			
6.5.4	GeneratingFunctions	18			
7	Strings	19			
7.1	String Matching	19			
7.1.1	KMP	19			
7.1.2	RabinKarp	19			
7.1.3	ZFunction	19			
7.2	Hashing	19			
7.2.1	StringHashing	19			
7.3	Aho Corasick	19			
7.3.1	AhoCorasick	19			
7.4	Suffix Structures	20			
7.4.1	SuffixArray	20			
7.4.2	SuffixAutomaton	20			
7.5	Palindromes	20			
7.5.1	Manacher	20			
8	Dynamic Programming (DP)	21			
8.1	DP Optimizations	21			
8.1.1	DivideAndConquerDP	21			
8.1.2	CHT	21			
8.1.3	CHTRollback	21			
8.1.4	LiChaoTree	22			
8.1.5	PersistentLiChaoTree	22			
8.1.6	KnuthDP	22			
8.1.7	AliensTrick	23			
8.2	Bitmask DP	23			
8.2.1	SOS_DP	23			
9	Trees	23			

9.1	Lowest Common Ancestor (LCA)	23
9.1.1	LCA	23
9.2	DSU on Tree	23
9.2.1	DSUOnTree	23
9.3	Heavy Light Decomposition (HLD)	24
9.3.1	HLD	24
9.4	Centroid Decomposition	24
9.4.1	Centroid	24
9.5	Tree Diameter	24
9.5.1	TreeDiameter	24
9.6	Euler Tour	25
9.6.1	EulerTour	25

10	Game Theory	25
----	-------------	----

11	Geometry	25
11.1	Geometry 2D	25
11.1.1	Point2D	25
11.1.2	Line2D	25
11.1.3	AngleOperations	26
11.1.4	PolygonArea	26
11.1.5	Circle3Points	26
11.1.6	CircleLineIntersection	26
11.1.7	CircleCircleIntersection	26
11.1.8	ConvexHull	26
11.1.9	RotatingCalipers	26
11.1.10	MinimumEnclosingCircle	26
11.1.11	HalfPlaneIntersection	27
11.2	Geometry 3D	27
11.2.1	Point3D	27

1 Miscellaneous

1.1 Template

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 // #define int long long
6 #define ll long long
7 #define ld long double
8 #define fi first
9 #define se second
10 #define pb push_back
11 #define eb emplace_back
12 #define all(x) x.begin(), x.end()
13 #define rall(x) x.rbegin(), x.rend()
14 #define ones(n) __builtin_popcountll(n)
15 #define msb(n) (63 - __builtin_clzll(n))
16 #define lsb(n) __builtin_ctzll(n)
17
18 const int N = 2e5 + 5, M = 1e3 + 5, LOG = 31;
19 const int inf = 0x3f3f3f3f;
20 const ll llinf = 0x3f3f3f3f3f3f3f3f;
21 const int MOD = 1e9 + 7;
22 const double eps = 1e-9, PI = acos(-1);
23
24 void testCase() {
25 }
26
27
28 void preCompute() {
29 }
30
31 int32_t main() {
32 #ifdef C_Lion
33     freopen("input.txt", "r", stdin);
34     freopen("output.txt", "w", stdout);
35     freopen("errors.txt", "w", stderr);
36 #else
37     //     freopen("input.in", "r", stdin);
38     //     freopen("output.out", "w", stdout);
39 #endif
40
41     ios_base::sync_with_stdio(false), cin.tie(nullptr), cout.tie(
        nullptr);
42     cout << fixed << setprecision(static_cast<int32_t>(-log10(eps)));
43     preCompute();
44
45     int tc = 1;
46     cin >> tc;
47     for (int TC = 1; TC <= tc; TC++) {
48         // cout << "Case #" << TC << ": ";
49         testCase();
50     }
51 }
52
53 /*
54
55
56
57
58 */
```

1.2 fastIO

```
1 const int MOD = 1e9 + 7;
2 const int BUF_SZ = 1 << 15;
3
4 inline namespace Input {
5     char buf[BUF_SZ];
6     int pos;
7     int len;
8     char next_char() {
9         if (pos == len) {
10             pos = 0;
11             len = (int)fread(buf, 1, BUF_SZ, stdin);
12             if (!len) { return EOF; }
13         }
14         return buf[pos++];
15     }
16
17     int read_int() {
18         int x;
19         char ch;
20         int sgn = 1;
21         while (!isdigit(ch = next_char())) {
22             if (ch == '-') { sgn *= -1; }
23         }
24         x = ch - '0';
25         while (isdigit(ch = next_char())) { x = x * 10 + (ch - '0'); }
26         return x * sgn;
27     }
28 } // namespace Input
29 inline namespace Output {
30     char buf[BUF_SZ];
31     int pos;
32
33     void flush_out() {
34         fwrite(buf, 1, pos, stdout);
35         pos = 0;
36     }
37
38     void write_char(char c) {
39         if (pos == BUF_SZ) { flush_out(); }
40         buf[pos++] = c;
41     }
42
43     void write_int(int x) {
44         static char num_buf[100];
45         if (x < 0) {
46             write_char('-');
47             x *= -1;
48         }
49         int len = 0;
50         for (; x >= 10; x /= 10) { num_buf[len++] = (char)('0' + (x % 10)); }
51         write_char((char)('0' + x));
52         while (len) { write_char(num_buf[--len]); }
53         write_char('\n');
54     }
55
56     // auto-flush output when program exits
57     void init_output() { assert(atexit(flush_out) == 0); }
58 } // namespace Output
```

1.3 grid

```
1 int dx[] = {0, 0, -1, 1, -1, -1, 1, 1};
2 int dy[] = {-1, 1, 0, 0, -1, 1, -1, 1};
3 char di[] = {'L', 'R', 'U', 'D'};
```

1.4 random

```
1 mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
2 template <typename T>
3 T random(T l, T r) {
4     return uniform_int_distribution(l, r)(rng);
5 }
6
7 vector<int> random(int n) {
8     vector<int> v(n);
9     // shuffle 1
10    shuffle(v.begin(), v.end(), rng);
11    // shuffle 2
12    for (int i = 1; i < n; i++)
13        swap(v[i], v[uniform_int_distribution<int>(0, i)(rng)]);
14    return v;
15 }
```

1.5 coordinateCompression

```
1 // Coordinate Compression - O(N log N)
2 template<typename T>
3 struct Compress {
4     vector<T> v;
5
6     Compress(vector<T>& a) : v(a) {
7         sort(all(v));
8         v.erase(unique(all(v)), v.end());
9         for (auto &x : a) {
10             x = lower_bound(all(v), x) - v.begin();
11         }
12     }
13
14     T operator[](int i) const {
15         return v[i];
16     }
17 };
```

1.6 grayCode

```
1 // Gray Code Generator - O(2^N)
2 // g(n) = n ^ (n >> 1)
3 vector<int> grayCode(int n) {
4     vector<int> res(1 << n);
```

```

5     for (int i = 0; i < (1 << n); i++) {
6         res[i] = i ^ (i >> 1);
7     }
8     return res;
9 }

```

1.7 customHashHashTable

```

1 #include <ext/pb_ds/assoc_container.hpp>
2 using namespace __gnu_pbds;
3
4 // Custom Hash to prevent anti-hash tests in unordered_map /
5 // gp_hash_table
6 struct custom_hash {
7     static uint64_t splitmix64(uint64_t x) {
8         x += 0x9e3779b97f4a7c15;
9         x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
10        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
11        return x ^ (x >> 31);
12    }
13
14    size_t operator()(uint64_t x) const {
15        static const uint64_t FIXED_RANDOM = chrono::steady_clock::now
16        ().time_since_epoch().count();
17        return splitmix64(x + FIXED_RANDOM);
18    }
19 };
20
21 // Usage: gp_hash_table<ll, int, custom_hash> table;
22 // unordered_map<ll, int, custom_hash> safe_map;

```

1.8 pragmas

```

1 // GCC Optimizations
2 #pragma GCC optimize("O3,unroll-loops")
3 #pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")

```

1.9 int128

```

1 // __int128 Stream Operators Helper
2 typedef __int128_t int128;
3 typedef __uint128_t uint128;
4
5 ostream &operator<<(ostream &os, int128 n) {
6     if (n == 0) return os << "0";
7     if (n < 0) { os << "-"; n = -n; }
8     string s;
9     while (n > 0) { s += (char)('0' + (n % 10)); n /= 10; }
10    reverse(all(s));
11    return os << s;
12 }
13
14 istream &operator>>(istream &is, int128 &n) {
15     string s; is >> s;
16     n = 0; int i = 0, sign = 1;
17     if (!s.empty() && s[0] == '-' ) { sign = -1; i = 1; }
18     for (; i < (int)s.size(); i++) n = n * 10 + (s[i] - '0');
19     n *= sign;
20     return is;
21 }

```

2 Data Structures (DS)

2.1 Segment Tree

2.1.1 SegmentTree

```

1 struct SEG {
2     ll sum = 0;
3     SEG() {
4     }
5     SEG(ll x){
6         sum = x;
7     }
8 };
9 struct segTree {
10    vector<SEG> seg;
11    int sz = 1,n;
12    segTree(int nn){
13        n = nn;
14        while (sz < nn)
15            sz *= 2;
16        seg.assign(2 * sz , SEG());
17    }
18
19    SEG merge(SEG seg1, SEG seg2) {
20        SEG ret;
21        ret.sum = seg1.sum + seg2.sum;
22        return ret;
23    }
24
25    void build(vector<int> &v,int x, int lx, int rx) {
26        if (lx == rx) {
27            seg[x] = SEG(v[lx]);
28            return;
29        }
30        int mid = (lx + rx) / 2;
31        int lf = 2 * x + 1, rt = 2 * x + 2;
32
33        build(v,lf, lx, mid);
34        build(v,rt, mid + 1, rx);
35        seg[x] = merge(seg[lf], seg[rt]);
36    }
37    void build(vector<int> &v){
38        build(v,0, 0, n-1);
39    }
40
41    void update(int i, ll val, int x, int lx, int rx) {
42        if (lx == rx) {
43            seg[x] = SEG(val);
44            return;
45        }

```

```

46        int mid = (lx + rx) / 2,lf = 2*x+1,rt= 2*x+2;
47        if(i <= mid)
48            update(i, val, lf, lx, mid);
49        else
50            update(i, val, rt, mid + 1, rx);
51        seg[x] = merge(seg[lf], seg[rt]);
52    }
53
54    void update(int i, ll val) {
55        update(i, val, 0, 0, n-1);
56    }
57
58    SEG query(int l, int r, int x, int lx, int rx) {
59        if (l <= lx && r >= rx)
60            return seg[x];
61        if (l > rx || r < lx)
62            return SEG();
63
64        int mid = (lx + rx) / 2,lf = 2 * x + 1,rt = 2*x+2;
65
66        return merge(query(l, r, lf, lx, mid) , query(l, r, rt, mid +
67        1, rx));
68    }
69
70    SEG query(int l, int r) {
71        return query(l, r, 0, 0, n-1);
72    }
73 };

```

2.1.2 LazySegmentTree

```

1 struct SEG {
2     ll sum = 0, lz = 0;
3
4     SEG() {
5     }
6
7     SEG(ll x) {
8         sum = x;
9     }
10 };
11
12 struct segTree {
13    vector<SEG> seg;
14    int sz = 1, n;
15    ll NO_OP = 0;
16
17    segTree(int nn) {
18        n = nn;
19        while (sz < nn)
20            sz *= 2;
21        seg.assign(2 * sz, SEG());
22    }
23
24    SEG merge(SEG seg1, SEG seg2) {
25        SEG ret;
26        ret.sum = seg1.sum + seg2.sum;
27        return ret;
28    }
29
30    void build(vector<int> &v, int x, int lx, int rx) {
31        if (lx == rx) {
32            seg[x] = SEG(v[lx]);
33            return;
34        }
35        int mid = (lx + rx) / 2;
36        int lf = 2 * x + 1, rt = 2 * x + 2;
37
38        build(v, lf, lx, mid);
39        build(v, rt, mid + 1, rx);
40        seg[x] = merge(seg[lf], seg[rt]);
41    }
42
43    void build(vector<int> &v) {
44        build(v, 0, 0, n - 1);
45    }
46
47    void push(int x, int lx, int rx) {
48        if (seg[x].lz != NO_OP) {
49            seg[x].sum += seg[x].lz * (rx - lx + 1);
50            int lf = 2 * x + 1, rt = 2 * x + 2;
51            if (lx != rx) {
52                seg[lf].lz += seg[x].lz;
53                seg[rt].lz += seg[x].lz;
54            }
55            seg[x].lz = NO_OP;
56        }
57    }
58
59    void update(int l, int r, ll val, int x, int lx, int rx) {
60        push(x, lx, rx);
61        if (l > rx || r < lx)
62            return;
63        if (l <= lx && r >= rx) {
64            seg[x].lz += val;
65            push(x, lx, rx);
66            return;
67        }
68        int mid = (lx + rx) / 2, lf = 2 * x + 1, rt = 2 * x + 2;
69        update(l, r, val, lf, lx, mid);
70        update(l, r, val, rt, mid + 1, rx);
71        seg[x] = merge(seg[lf], seg[rt]);
72    }
73
74    void update(int l, int r, ll val) {
75        update(l, r, val, 0, 0, sz - 1);
76    }
77
78    SEG query(int l, int r, int x, int lx, int rx) {
79        push(x, lx, rx);
80        if (l <= lx && r >= rx)
81            return seg[x];
82        if (l > rx || r < lx)
83            return SEG();

```

```

84         int mid = (lx + rx) / 2, lf = 2 * x + 1, rt = 2 * x + 2;
85
86         return merge(query(l, r, lf, lx, mid), query(l, r, rt, mid + 1,
87 rx));
88     }
89
90     SEG query(int l, int r) {
91         return query(l, r, 0, 0, sz - 1);
92     }
93 };

```

2.1.3 PersistentSegmentTree

```

1 struct Node {
2     Node *l, *r;
3     ll val = 0;
4
5     Node(int val) : l(NULL), r(NULL), val(val) {
6     }
7
8     Node() : l(NULL), r(NULL) {
9     }
10
11     Node(Node* l, Node* r) : l(l), r(r) {
12         if (l != NULL) val += l->val;
13         if (r != NULL) val += r->val;
14     }
15
16     void addChild() {
17         l = new Node();
18         r = new Node();
19     }
20 };
21
22 struct PST {
23     int n;
24
25     PST(int n) : n(n + 1) {
26     }
27
28     Node merge(Node x, Node y) {
29         Node ret;
30         ret.val = x.val + y.val;
31         return ret;
32     }
33
34     Node* update(Node* v, int i, int val, int lx, int rx) {
35         if (lx == rx) return new Node(v->val + val);
36         int mid = (lx + rx) / 2;
37         if (!v->l) v->addChild();
38         if (i <= mid) return new Node(update(v->l, i, val, lx, mid), v
39 ->r);
40         return new Node(v->l, update(v->r, i, val, mid + 1, rx));
41     }
42
43     Node* update(Node* v, int i, int val) { return update(v, i, val, 0,
44 n - 1); }
45
46     Node query(Node* v, int l, int r, int lx, int rx) {
47         if (l > rx || r < lx) return {};
48         if (l <= lx && r >= rx) return *v;
49         if (!v->l) v->addChild();
50         int mid = (lx + rx) / 2;
51         return merge(query(v->l, l, r, lx, mid), query(v->r, l, r, mid
52 + 1, rx));
53     }
54
55     Node query(Node* v, int l, int r) { return query(v, l, r, 0, n - 1)
56 ; }
57
58     int getKth(Node* a, Node* b, int k, int lx, int rx) {
59         if (lx == rx) return lx;
60         if (!a->l) a->addChild();
61         if (!b->l) b->addChild();
62         int rem = b->l->val - a->l->val;
63         int mid = (lx + rx) / 2;
64         if (rem >= k) return getKth(a->l, b->l, k, lx, mid);
65         return getKth(a->r, b->r, k - rem, mid + 1, rx);
66     }
67
68     int getKth(Node* a, Node* b, int k) { return getKth(a, b, k, 0, n -
69 1); }
70 };

```

2.1.4 PersistentSegTreeVector

```

1 struct Node {
2     Node *l, *r;
3     int val = 0;
4
5     Node(int val): l(NULL), r(NULL), val(val) {}
6     Node(): l(NULL), r(NULL) {}
7     Node(Node *l, Node *r): l(l), r(r) {
8         if (l != NULL) val += l->val;
9         if (r != NULL) val += r->val;
10    }
11
12    void addChild() {
13        l = new Node();
14        r = new Node();
15    }
16 };
17
18 struct PST {
19     int n;
20     PST(int n): n(n + 1) {}
21
22     Node merge(Node x, Node y) {
23         Node ret;
24         ret.val = x.val + y.val;
25         return ret;
26     }
27

```

```

28     Node *set(Node *v, int i, int val, int lx, int rx) {
29         if (lx == rx) return new Node(val);
30         int mid = (lx + rx) / 2;
31         if (!v->l) v->addChild();
32         if (i <= mid) return new Node(set(v->l, i, val, lx, mid), v->r)
33 ;
34         return new Node(v->l, set(v->r, i, val, mid + 1, rx));
35     }
36
37     Node *set(Node *v, int i, int val) { return set(v, i, val, 0, n -
38 1); }
39
40     // [l, r] r is included
41     Node query(Node *v, int l, int r, int lx, int rx) {
42         if (l > rx || r < lx) return {};
43         if (l <= lx && r >= rx) return *v;
44         if (!v->l) v->addChild();
45         int mid = (lx + rx) / 2;
46         return merge(query(v->l, l, r, lx, mid), query(v->r, l, r, mid
47 + 1, rx));
48     }
49
50     Node query(Node *v, int l, int r) { return query(v, l, r, 0, n - 1)
51 ; }
52
53     int getKth(Node *a, Node *b, int k, int lx, int rx) {
54         if (lx == rx) return lx;
55         if (!a->l) a->addChild();
56         if (!b->l) b->addChild();
57         int rem = b->l->val - a->l->val;
58         int mid = (lx + rx) / 2;
59         if (rem >= k) return getKth(a->l, b->l, k, lx, mid);
60         return getKth(a->r, b->r, k - rem, mid + 1, rx);
61     }
62
63     int getKth(Node *a, Node *b, int k) { return getKth(a, b, k, 0, n -
64 1); }
65 };

```

2.1.5 SegmentTreeBeats

```

1 struct Node {
2     int mn;
3     int mnCnt;
4     int secondMn;
5
6     int mx;
7     int mxCnt;
8     int secondMx;
9
10    int sum;
11
12    int lazyAdd;
13    int lazyEqual;
14    bool isLazyEqual;
15
16    Node() {
17        mx = -inf;
18        secondMx = -inf;
19        mxCnt = 0;
20
21        mn = inf;
22        secondMn = inf;
23        mnCnt = 0;
24
25        sum = 0;
26
27        lazyAdd = 0;
28        lazyEqual = 0;
29        isLazyEqual = 0;
30    };
31
32    Node(int x) {
33        mx = x;
34        secondMx = -inf;
35        mxCnt = 1;
36
37        mn = x;
38        secondMn = inf;
39        mnCnt = 1;
40
41        sum = x;
42
43        lazyAdd = 0;
44        lazyEqual = 0;
45        isLazyEqual = 0;
46    }
47 };
48
49 struct SegTree {
50     int tree_size;
51     vector<Node> seg_data;
52
53     SegTree(int n) {
54         tree_size = 1;
55         while (tree_size < n) tree_size *= 2;
56         seg_data.resize(2 * tree_size, Node());
57     }
58
59     Node merge(Node& lf, Node& ri) {
60         Node ans = Node();
61
62         ans.sum = lf.sum + ri.sum;
63
64         ans.mx = max(lf.mx, ri.mx);
65         ans.secondMx = max(lf.secondMx, ri.secondMx);
66         ans.mxCnt = 0;
67         if (lf.mx == ans.mx) ans.mxCnt += lf.mxCnt;
68         else ans.secondMx = max(ans.secondMx, lf.mx);
69         if (ri.mx == ans.mx) ans.mxCnt += ri.mxCnt;
70         else ans.secondMx = max(ans.secondMx, ri.mx);
71
72         ans.mn = min(lf.mn, ri.mn);
73         ans.secondMn = min(lf.secondMn, ri.secondMn);

```

```

74     ans.mnCnt = 0;
75     if (lf.mn == ans.mn) ans.mnCnt += lf.mnCnt;
76     else ans.secondMn = min(ans.secondMn, lf.mn);
77     if (ri.mn == ans.mn) ans.mnCnt += ri.mnCnt;
78     else ans.secondMn = min(ans.secondMn, ri.mn);
79
80     return ans;
81 }
82
83 void init(vector<int>& nums, int ni, int lx, int rx) {
84     if (rx - lx == 1) {
85         if (lx < nums.size())
86             seg_data[ni] = Node(nums[lx]);
87         return;
88     }
89
90     int mid = lx + (rx - lx) / 2;
91     init(nums, 2 * ni + 1, lx, mid);
92     init(nums, 2 * ni + 2, mid, rx);
93
94     seg_data[ni] = merge(seg_data[2 * ni + 1], seg_data[2 * ni +
95     2]);
96 }
97
98 void init(vector<int>& nums) {
99     init(nums, 0, 0, tree_size);
100 }
101
102 void doPushEq(int ni, int lx, int rx, int x) {
103     seg_data[ni].mn = seg_data[ni].mx = seg_data[ni].lazyEqual = x;
104     seg_data[ni].isLazyEqual = 1;
105     seg_data[ni].mnCnt = seg_data[ni].mxCnt = rx - lx;
106     seg_data[ni].secondMx = -inf;
107     seg_data[ni].secondMn = inf;
108
109     seg_data[ni].sum = x * (rx - lx);
110     seg_data[ni].lazyAdd = 0;
111 }
112
113 void doPushMinEq(int ni, int lx, int rx, int x) {
114     if (seg_data[ni].mn >= x) doPushEq(ni, lx, rx, x);
115     else if (seg_data[ni].mx > x) {
116         if (seg_data[ni].secondMn == seg_data[ni].mx)
117             seg_data[ni].secondMn = x;
118         seg_data[ni].sum -= (seg_data[ni].mx - x) * seg_data[ni].
119             mxCnt;
120         seg_data[ni].mx = x;
121     }
122 }
123
124 void doPushMaxEq(int ni, int lx, int rx, int x) {
125     if (seg_data[ni].mx <= x) doPushEq(ni, lx, rx, x);
126     else if (seg_data[ni].mn < x) {
127         if (seg_data[ni].secondMx == seg_data[ni].mn)
128             seg_data[ni].secondMx = x;
129
130         seg_data[ni].sum += (x - seg_data[ni].mn) * seg_data[ni].
131             mnCnt;
132         seg_data[ni].mn = x;
133     }
134 }
135
136 void doPushSum(int ni, int lx, int rx, int x) {
137     if (seg_data[ni].mn == seg_data[ni].mx)
138         doPushEq(ni, lx, rx, seg_data[ni].mn + x);
139     else {
140         seg_data[ni].mx += x;
141         if (seg_data[ni].secondMx != -inf)
142             seg_data[ni].secondMx += x;
143
144         seg_data[ni].mn += x;
145         if (seg_data[ni].secondMn != inf)
146             seg_data[ni].secondMn += x;
147
148         seg_data[ni].sum += (rx - lx) * x;
149         seg_data[ni].lazyAdd += x;
150     }
151 }
152
153 void propagete(int ni, int lx, int rx) {
154     if (rx - lx == 1) return;
155
156     int mid = lx + (rx - lx) / 2;
157     if (seg_data[ni].isLazyEqual) {
158         doPushEq(2 * ni + 1, lx, mid, seg_data[ni].lazyEqual);
159         doPushEq(2 * ni + 2, mid, rx, seg_data[ni].lazyEqual);
160         seg_data[ni].lazyEqual = seg_data[ni].isLazyEqual = 0;
161     }
162     else {
163         doPushSum(2 * ni + 1, lx, mid, seg_data[ni].lazyAdd);
164         doPushSum(2 * ni + 2, mid, rx, seg_data[ni].lazyAdd);
165         seg_data[ni].lazyAdd = 0;
166
167         doPushMinEq(2 * ni + 1, lx, mid, seg_data[ni].mx);
168         doPushMinEq(2 * ni + 2, mid, rx, seg_data[ni].mx);
169
170         doPushMaxEq(2 * ni + 1, lx, mid, seg_data[ni].mn);
171         doPushMaxEq(2 * ni + 2, mid, rx, seg_data[ni].mn);
172     }
173 }
174
175 void minimize(int l, int r, int x, int ni, int lx, int rx) {
176     if (rx <= l || lx >= r || seg_data[ni].mx <= x)
177         return;
178     if (lx >= l && rx <= r && seg_data[ni].secondMx < x) {
179         doPushMinEq(ni, lx, rx, x);
180         return;
181     }
182
183     propagete(ni, lx, rx);
184
185     int mid = lx + (rx - lx) / 2;
186     minimize(l, r, x, 2 * ni + 1, lx, mid);
187     minimize(l, r, x, 2 * ni + 2, mid, rx);

```

```

185     seg_data[ni] = merge(seg_data[2 * ni + 1], seg_data[2 * ni +
186     2]);
187 }
188
189 void minimize(int l, int r, int x) {
190     minimize(l, r, x, 0, 0, tree_size);
191 }
192
193 void maximize(int l, int r, int x, int ni, int lx, int rx) {
194     if (rx <= l || lx >= r || seg_data[ni].mn >= x)
195         return;
196     if (lx >= l && rx <= r && seg_data[ni].secondMn > x) {
197         doPushMaxEq(ni, lx, rx, x);
198         return;
199     }
200
201     propagete(ni, lx, rx);
202
203     int mid = lx + (rx - lx) / 2;
204     maximize(l, r, x, 2 * ni + 1, lx, mid);
205     maximize(l, r, x, 2 * ni + 2, mid, rx);
206
207     seg_data[ni] = merge(seg_data[2 * ni + 1], seg_data[2 * ni +
208     2]);
209 }
210
211 void maximize(int l, int r, int x) {
212     maximize(l, r, x, 0, 0, tree_size);
213 }
214
215 void add(int l, int r, int x, int ni, int lx, int rx) {
216     if (rx <= l || lx >= r)
217         return;
218     if (lx >= l && rx <= r) {
219         doPushSum(ni, lx, rx, x);
220         return;
221     }
222
223     propagete(ni, lx, rx);
224
225     int mid = lx + (rx - lx) / 2;
226     add(l, r, x, 2 * ni + 1, lx, mid);
227     add(l, r, x, 2 * ni + 2, mid, rx);
228
229     seg_data[ni] = merge(seg_data[2 * ni + 1], seg_data[2 * ni +
230     2]);
231 }
232
233 void add(int l, int r, int x) {
234     add(l, r, x, 0, 0, tree_size);
235 }
236
237 void set(int l, int r, int x, int ni, int lx, int rx) {
238     if (rx <= l || lx >= r)
239         return;
240     if (lx >= l && rx <= r) {
241         doPushEq(ni, lx, rx, x);
242         return;
243     }
244
245     propagete(ni, lx, rx);
246
247     int mid = lx + (rx - lx) / 2;
248     set(l, r, x, 2 * ni + 1, lx, mid);
249     set(l, r, x, 2 * ni + 2, mid, rx);
250
251     seg_data[ni] = merge(seg_data[2 * ni + 1], seg_data[2 * ni +
252     2]);
253 }
254
255 void set(int l, int r, int x) {
256     set(l, r, x, 0, 0, tree_size);
257 }
258
259 Node get_range(int l, int r, int ni, int lx, int rx) {
260     propagete(ni, lx, rx);
261
262     if (lx >= l && rx <= r)
263         return seg_data[ni];
264     if (rx <= l || lx >= r)
265         return Node();
266
267     int mid = (lx + rx) / 2;
268
269     Node lf = get_range(l, r, 2 * ni + 1, lx, mid);
270     Node ri = get_range(l, r, 2 * ni + 2, mid, rx);
271     return merge(lf, ri);
272 }
273
274 int get_sum(int l, int r) {
275     return get_range(l, r, 0, 0, tree_size).sum;
276 }
277
278 int get_mx(int l, int r) {
279     return get_range(l, r, 0, 0, tree_size).mx;
280 }
281
282 int get_mn(int l, int r) {
283     return get_range(l, r, 0, 0, tree_size).mn;
284 }
285
286 };

```

2.1.6 DynamicSegTree

```

1 // Dynamic (Implicit) Segment Tree - Space: O(Q log N) per query
2 // Allocates nodes on demand for ranges up to 1e9 or 1e18
3 template<typename T = ll>
4 struct DynamicSegTree {
5     struct Node {
6         T val = 0;
7         int lc = -1, rc = -1;
8     };
9

```

```

10     vector<Node> tree;
11     ll L, R;
12
13     DynamicSegTree(ll L = 0, ll R = 1e9) : L(L), R(R) {
14         add_node();
15     }
16
17     int add_node() {
18         tree.pb(Node());
19         return tree.size() - 1;
20     }
21
22     void update(int node, ll l, ll r, ll pos, T val) {
23         tree[node].val += val;
24         if (l == r) return;
25         ll mid = l + (r - l) / 2;
26         if (pos <= mid) {
27             if (tree[node].lc == -1) {
28                 int nxt = add_node();
29                 tree[node].lc = nxt;
30             }
31             update(tree[node].lc, l, mid, pos, val);
32         } else {
33             if (tree[node].rc == -1) {
34                 int nxt = add_node();
35                 tree[node].rc = nxt;
36             }
37             update(tree[node].rc, mid + 1, r, pos, val);
38         }
39     }
40
41     void update(ll pos, T val) {
42         update(0, L, R, pos, val);
43     }
44
45     T query(int node, ll l, ll r, ll ql, ll qr) {
46         if (node == -1 || ql > r || qr < l) return 0;
47         if (ql <= l && r <= qr) return tree[node].val;
48         ll mid = l + (r - l) / 2;
49         return query(tree[node].lc, l, mid, ql, qr) + query(tree[node].
50             rc, mid + 1, r, ql, qr);
51     }
52
53     T query(ll ql, ll qr) {
54         return query(0, L, R, ql, qr);
55     }
56 };

```

2.1.7 MergeSortTree

```

1 // Merge Sort Tree - Space: O(N log N), Build: O(N log N), Query: O(log
2 // Stores sorted elements in each node to answer 2D range count queries
3 template<typename T = int>
4 struct MergeSortTree {
5     int n;
6     vector<vector<T>> tree;
7
8     MergeSortTree(const vector<T>& a) : n(a.size()), tree(4 * n) {
9         build(1, 0, n - 1, a);
10    }
11
12    void build(int node, int l, int r, const vector<T>& a) {
13        if (l == r) {
14            tree[node] = {a[l]};
15            return;
16        }
17        int mid = (l + r) / 2;
18        build(2 * node, l, mid, a);
19        build(2 * node + 1, mid + 1, r, a);
20        merge(all(tree[2 * node]), all(tree[2 * node + 1]),
21            back_inserter(tree[node]));
22    }
23
24    // Returns number of elements <= k in index range [ql, qr]
25    int query_count_le(int node, int l, int r, int ql, int qr, T k) {
26        if (ql > r || qr < l) return 0;
27        if (ql <= l && r <= qr) {
28            return upper_bound(all(tree[node]), k) - tree[node].begin()
29                ;
30        }
31        int mid = (l + r) / 2;
32        return query_count_le(2 * node, l, mid, ql, qr, k) +
33            query_count_le(2 * node + 1, mid + 1, r, ql, qr, k);
34    }
35
36    int query_count_le(int ql, int qr, T k) {
37        return query_count_le(1, 0, n - 1, ql, qr, k);
38    }
39 };

```

2.1.8 IterativeSegTree

```

1 // Iterative (Non-Recursive) Segment Tree - Space: O(N), Update: O(log
2 // Fast, cache-friendly implementation for point update and range query
3 template<typename T = ll>
4 struct IterativeSegTree {
5     int n;
6     vector<T> tree;
7     T neutral;
8     function<T(T, T)> combine;
9
10    IterativeSegTree(int n = 0, T neutral = 0, function<T(T, T)>
11        combine = [](T a, T b) { return a + b; })
12        : n(n), tree(2 * n, neutral), neutral(neutral), combine(combine
13            ) {}
14
15    IterativeSegTree(const vector<T>& a, T neutral = 0, function<T(T, T)
16        > combine = [](T a, T b) { return a + b; })
17        : n(a.size()), tree(2 * a.size(), neutral), neutral(neutral),
18            combine(combine) {
19        for (int i = 0; i < n; i++) tree[n + i] = a[i];

```

```

16         for (int i = n - 1; i > 0; i--) tree[i] = combine(tree[i << 1],
17             tree[i << 1 | 1]);
18     }
19
20     void update(int p, T val) {
21         for (tree[p += n] = val; p > 1; p >>= 1) {
22             tree[p >> 1] = combine(tree[p], tree[p ^ 1]);
23         }
24     }
25
26     // Range query on [l, r] inclusive
27     T query(int l, int r) {
28         T resl = neutral, resr = neutral;
29         for (l += n, r += n + 1; l < r; l >>= 1, r >>= 1) {
30             if (l & 1) resl = combine(resl, tree[l++]);
31             if (r & 1) resr = combine(tree[--r], resr);
32         }
33         return combine(resl, resr);
34     };

```

2.1.9 SegTree2D

```

1 // 2D Segment Tree - Space: O(N * M), Update: O(log N * log M), Query:
2 // Supports point updates and 2D subgrid range sum queries
3 template<typename T = ll>
4 struct SegTree2D {
5     int n, m;
6     vector<vector<T>> tree;
7
8     SegTree2D(int n = 0, int m = 0) : n(n), m(m), tree(4 * n, vector<T>
9        >(4 * m, 0)) {}
10
11    void update_y(int vx, int lx, int rx, int ry, int ly, int ry, int x
12        , int y, T val) {
13        if (ly == ry) {
14            if (lx == rx) tree[vx][vy] += val;
15            else tree[vx][vy] = tree[2 * vx][vy] + tree[2 * vx + 1][vy
16                ];
17            return;
18        }
19        int my = (ly + ry) / 2;
20        if (y <= my) update_y(vx, lx, rx, 2 * ry, ly, my, x, y, val);
21        else update_y(vx, lx, rx, 2 * ry + 1, my + 1, ry, x, y, val);
22        tree[vx][vy] = tree[vx][2 * ry] + tree[vx][2 * ry + 1];
23    }
24
25    void update_x(int vx, int lx, int rx, int x, int y, T val) {
26        if (lx != rx) {
27            int mx = (lx + rx) / 2;
28            if (x <= mx) update_x(2 * vx, lx, mx, x, y, val);
29            else update_x(2 * vx + 1, mx + 1, rx, x, y, val);
30        }
31        update_y(vx, lx, rx, 1, 0, m - 1, x, y, val);
32    }
33
34    void add(int x, int y, T val) {
35        update_x(1, 0, n - 1, x, y, val);
36    }
37
38    T query_y(int vx, int vy, int ly, int ry, int qy1, int qy2) {
39        if (qy1 > ry || qy2 < ly) return 0;
40        if (qy1 <= ly && ry <= qy2) return tree[vx][vy];
41        int my = (ly + ry) / 2;
42        return query_y(vx, 2 * vy, ly, my, qy1, qy2) + query_y(vx, 2 *
43            vy + 1, my + 1, ry, qy1, qy2);
44    }
45
46    T query_x(int vx, int lx, int rx, int qx1, int qx2, int qy1, int
47        qy2) {
48        if (qx1 > rx || qx2 < lx) return 0;
49        if (qx1 <= lx && rx <= qx2) return query_y(vx, 1, 0, m - 1, qy1
50            , qy2);
51        int mx = (lx + rx) / 2;
52        return query_x(2 * vx, lx, mx, qx1, qx2, qy1, qy2) + query_x(2
53            * vx + 1, mx + 1, rx, qx1, qx2, qy1, qy2);
54    }
55
56    T query(int x1, int y1, int x2, int y2) {
57        return query_x(1, 0, n - 1, x1, x2, y1, y2);
58    }
59 };

```

2.2 Binary Indexed Tree (BIT)

2.2.1 FenwickTree

```

1 // Fenwick Tree (Binary Indexed Tree) - 0-based indexing
2 // Space: O(N), Time: O(log N) per update/query
3 template<typename T = ll>
4 struct FenwickTree {
5     int n;
6     vector<T> tree;
7
8     FenwickTree(int n = 0) : n(n), tree(n + 1, 0) {}
9
10    void add(int i, T delta) {
11        for (i++; i <= n; i += i & -i) tree[i] += delta;
12    }
13
14    void set(int i, T val) {
15        add(i, val - query(i, i));
16    }
17
18    T query(int i) {
19        T sum = 0;
20        for (i++; i > 0; i -= i & -i) sum += tree[i];
21        return sum;
22    }
23
24    T query(int l, int r) {
25        if (l > r) return 0;
26        return query(r) - query(l - 1);

```



```

27     }
28
29     // Returns first 0-based index where prefix sum >= val
30     int lower_bound(T val) {
31         int pos = 0;
32         for (int sz = 1 << (n ? __lg(n) : 0); sz > 0; sz >>= 1) {
33             if (pos + sz <= n && tree[pos + sz] < val) {
34                 val -= tree[pos + sz];
35                 pos += sz;
36             }
37         }
38         return pos;
39     }
40 };

```

2.2.2 FenwickTree2D

```

1 // 2D Fenwick Tree - 0-based indexing
2 // Space: O(N * M), Time: O(log N * log M) per update/query
3 template<typename T = ll>
4 struct FenwickTree2D {
5     int n, m;
6     vector<vector<T>> tree;
7
8     FenwickTree2D(int n = 0, int m = 0) : n(n), m(m), tree(n + 1,
9         vector<T>(m + 1, 0)) {}
10
11     void add(int x, int y, T val) {
12         for (int i = x + 1; i <= n; i += i & -i) {
13             for (int j = y + 1; j <= m; j += j & -j) {
14                 tree[i][j] += val;
15             }
16         }
17     }
18
19     void set(int x, int y, T val) {
20         add(x, y, val - query(x, y, x, y));
21     }
22
23     T query(int x, int y) {
24         T sum = 0;
25         for (int i = min(x + 1, n); i > 0; i -= i & -i) {
26             for (int j = min(y + 1, m); j > 0; j -= j & -j) {
27                 sum += tree[i][j];
28             }
29         }
30         return sum;
31     }
32
33     // 2D Subgrid Range Sum from (x1, y1) to (x2, y2) inclusive
34     T query(int x1, int y1, int x2, int y2) {
35         if (x1 > x2 || y1 > y2) return 0;
36         return query(x2, y2) - query(x1 - 1, y2) - query(x2, y1 - 1) +
37             query(x1 - 1, y1 - 1);
38     }
39 };

```

2.2.3 Offline2DFenwickTree

```

1 // Offline 2D Fenwick Tree (Compressed 2D BIT)
2 // Space: O(N + Q log N), Time: O(log^2 N) per query
3 template<typename T = ll>
4 struct BIT2D {
5     int n;
6     vector<vector<int>> vals;
7     vector<vector<T>> bit;
8
9     int ind(const vector<int>& v, int x) {
10         return upper_bound(all(v), x) - v.begin() - 1;
11     }
12
13     // n: max row index, todo: all (r, c) update coordinates that will
14     // be called
15     BIT2D(int n, vector<array<int, 2>> todo) : n(n + 1), vals(n + 1),
16         bit(n + 1) {
17         sort(all(todo), [](const auto& a, const auto& b) { return a[1]
18             < b[1]; });
19
20         for (int i = 0; i <= n; i++) vals[i].pb(INT_MIN);
21         for (auto [r, c] : todo) {
22             for (int x = r; x <= n; x |= x + 1) {
23                 if (vals[x].back() != c) vals[x].pb(c);
24             }
25         }
26         for (int i = 0; i <= n; i++) bit[i].resize(vals[i].size(), 0);
27     }
28
29     void add(int r, int c, T val) {
30         for (; r <= n; r |= r + 1) {
31             for (int i = ind(vals[r], c); i < (int)bit[r].size(); i |=
32                 i + 1) {
33                 bit[r][i] += val;
34             }
35         }
36     }
37
38     T query(int r, int c) {
39         T res = 0;
40         for (; r >= 0; r = (r & (r + 1)) - 1) {
41             for (int i = ind(vals[r], c); i >= 0; i = (i & (i + 1)) -
42                 1) {
43                 res += bit[r][i];
44             }
45         }
46         return res;
47     }
48
49     T query(int r1, int c1, int r2, int c2) {
50         return query(r2, c2) - query(r2, c1 - 1) - query(r1 - 1, c2) +
51             query(r1 - 1, c1 - 1);
52     }
53
54     void reset() {
55         for (auto& v : bit) fill(all(v), 0);
56     }
57 };

```

```

50     }
51 };

```

2.3 Sparse Table

2.3.1 SparseTable

```

1 // Sparse Table (Range Minimum/Maximum Query) - O(N log N) build, O(1)
2 // query
3 template<typename T, typename F = function<T(T, T)>>
4 struct SparseTable {
5     int n;
6     vector<vector<T>> mat;
7     F func;
8
9     SparseTable() = default;
10    SparseTable(const vector<T>& a, F f = [](T x, T y) { return max(x,
11        y); }) : n(a.size()), func(f) {
12        int k = n ? __lg(n) + 1 : 0;
13        mat.assign(k, a);
14        for (int j = 1; j < k; j++) {
15            for (int i = 0; i + (1 << j) <= n; i++) {
16                mat[j][i] = func(mat[j - 1][i], mat[j - 1][i + (1 << (j
17                    - 1))]);
18            }
19        }
20    }
21
22    T query(int l, int r) const {
23        int j = __lg(r - l + 1);
24        return func(mat[j][l], mat[j][r - (1 << j) + 1]);
25    }
26 };

```

2.3.2 SparseTable2D

```

1 // 2D Sparse Table - O(N * M * log N * log M) build, O(1) query
2 template<typename T = ll>
3 struct SparseTable2D {
4     int n, m;
5     // jmp[kr][kc][i][j] = max in 2^kr x 2^kc subgrid starting at (i, j)
6     vector<vector<vector<vector<T>>>> jmp;
7
8     SparseTable2D(const vector<vector<T>>& grid) {
9         n = grid.size();
10        if (!n) return;
11        m = grid[0].size();
12        if (!m) return;
13
14        int log_n = __lg(n) + 1, log_m = __lg(m) + 1;
15        jmp.assign(log_n, vector<vector<vector<T>>>(log_m, vector<
16            vector<T>>(n, vector<T>(m))));
17
18        for (int i = 0; i < n; i++) {
19            for (int j = 0; j < m; j++) {
20                jmp[0][0][i][j] = grid[i][j];
21            }
22        }
23
24        for (int kc = 1; kc < log_m; kc++) {
25            for (int i = 0; i < n; i++) {
26                for (int j = 0; j + (1 << kc) <= m; j++) {
27                    jmp[0][kc][i][j] = max(jmp[0][kc - 1][i][j], jmp
28                        [0][kc - 1][i][j + (1 << (kc - 1))]);
29                }
30            }
31        }
32
33        for (int kr = 1; kr < log_n; kr++) {
34            for (int kc = 0; kc < log_m; kc++) {
35                for (int i = 0; i + (1 << kr) <= n; i++) {
36                    for (int j = 0; j + (1 << kc) <= m; j++) {
37                        jmp[kr][kc][i][j] = max(jmp[kr - 1][kc][i][j],
38                            jmp[kr - 1][kc][i + (1 << (kr - 1))][j]);
39                    }
40                }
41            }
42        }
43    }
44
45    T query(int r1, int c1, int r2, int c2) const {
46        int kr = __lg(r2 - r1 + 1), kc = __lg(c2 - c1 + 1);
47        return max({jmp[kr][kc][r1][c1],
48            jmp[kr][kc][r1][c2 - (1 << kc) + 1],
49            jmp[kr][kc][r2 - (1 << kr) + 1][c1],
50            jmp[kr][kc][r2 - (1 << kr) + 1][c2 - (1 << kc) +
51                1]});
52    }
53 };

```

2.3.3 SquareSparseTable

```

1 // 2D Square Sparse Table - O(N * M * log(min(N, M))) build, O(1)
2 // square query
3 template<typename T = ll>
4 struct SquareSparseTable {
5     int n, m;
6     // jmp[k][i][j] = max of 2^k x 2^k square starting at (i, j)
7     vector<vector<vector<T>>>> jmp;
8
9     SquareSparseTable(const vector<vector<T>>& grid) {
10        n = grid.size();
11        if (!n) return;
12        m = grid[0].size();
13        if (!m) return;
14
15        int log_max = __lg(min(n, m)) + 1;
16        jmp.assign(log_max, vector<vector<T>>>(n, vector<T>(m)));
17
18        for (int i = 0; i < n; i++) {
19            for (int j = 0; j < m; j++) {
20                jmp[0][i][j] = grid[i][j];
21            }
22        }
23    }
24 };

```

```

20     }
21 }
22
23 for (int k = 1; k < log_max; k++) {
24     int pw = 1 << (k - 1);
25     for (int i = 0; i + (pw * 2) <= n; i++) {
26         for (int j = 0; j + (pw * 2) <= m; j++) {
27             jmp[k][i][j] = max({jmp[k - 1][i][j],
28                                 jmp[k - 1][i][j + pw],
29                                 jmp[k - 1][i + pw][j],
30                                 jmp[k - 1][i + pw][j + pw]});
31         }
32     }
33 }
34 }
35
36 // Queries max in square subgrid from (r1, c1) to (r2, c2)
37 T query(int r1, int c1, int r2, int c2) const {
38     int k = __lg(r2 - r1 + 1), pw = 1 << k;
39     return max({jmp[k][r1][c1],
40                jmp[k][r1][c2 - pw + 1],
41                jmp[k][r2 - pw + 1][c1],
42                jmp[k][r2 - pw + 1][c2 - pw + 1]});
43 }
44 };

```

2.4 Trie

2.4.1 StringTrie

```

1 // Prefix Trie for Strings
2 // Time: O(L) per insert/query, Space: O(N * L * Alphabet)
3 struct Trie {
4     vector<vector<int>> nxt;
5     vector<int> cnt, end_cnt;
6     int alpha;
7
8     Trie(int alpha = 26) : alpha(alpha) {
9         add_node();
10    }
11
12    int add_node() {
13        nxt.emplace_back(alpha, -1);
14        cnt.emplace_back(0);
15        end_cnt.emplace_back(0);
16        return nxt.size() - 1;
17    }
18
19    void insert(const string& s, int val = 1) {
20        int u = 0;
21        cnt[u] += val;
22        for (char c : s) {
23            int ch = c - 'a';
24            if (nxt[u][ch] == -1) nxt[u][ch] = add_node();
25            u = nxt[u][ch];
26            cnt[u] += val;
27        }
28        end_cnt[u] += val;
29    }
30
31    int count_prefix(const string& s) {
32        int u = 0;
33        for (char c : s) {
34            int ch = c - 'a';
35            if (nxt[u][ch] == -1) return 0;
36            u = nxt[u][ch];
37        }
38        return cnt[u];
39    }
40 };

```

2.4.2 BinaryTrie

```

1 // Binary Trie for 64-bit integers and XOR operations
2 // Time: O(BITS) per operation
3 struct BinaryTrie {
4     vector<array<int, 2>> nxt;
5     vector<int> cnt;
6     int max_bit;
7
8     BinaryTrie(int max_bit = 60) : max_bit(max_bit) {
9         add_node();
10    }
11
12    int add_node() {
13        nxt.push_back({-1, -1});
14        cnt.push_back(0);
15        return nxt.size() - 1;
16    }
17
18    void insert(ll x, int val = 1) {
19        int u = 0;
20        cnt[u] += val;
21        for (int i = max_bit; i >= 0; i--) {
22            int b = (x >> i) & 1;
23            if (nxt[u][b] == -1) nxt[u][b] = add_node();
24            u = nxt[u][b];
25            cnt[u] += val;
26        }
27    }
28
29    // Returns element in trie maximizing (x ^ element)
30    ll max_xor(ll x) const {
31        if (!cnt[0]) return -1e18;
32        int u = 0;
33        ll res = 0;
34        for (int i = max_bit; i >= 0; i--) {
35            int b = (x >> i) & 1;
36            int want = b ^ 1;
37            if (nxt[u][want] != -1 && cnt[nxt[u][want]] > 0) {
38                res |= (1LL << i);
39                u = nxt[u][want];
40            } else {
41                u = nxt[u][b];

```

```

42        }
43    }
44    return res;
45 }
46 };

```

2.4.3 PersistentTrie

```

1 // Persistent Binary Trie - Space: O(N * BITS), Time: O(BITS) per
2 // insert/query
3 // Supports versioning and range max XOR queries between versions [
4 // l_ver, r_ver]
5 struct PersistentTrie {
6     struct Node {
7         int cnt = 0;
8         array<int, 2> nxt = {-1, -1};
9     };
10
11     vector<Node> tree;
12     vector<int> roots;
13     int max_bit;
14
15     PersistentTrie(int max_bit = 30) : max_bit(max_bit) {
16         roots.pb(add_node());
17     }
18
19     int add_node(int old_node = -1) {
20         if (old_node != -1) {
21             tree.pb(tree[old_node]);
22         } else {
23             tree.pb(Node());
24         }
25         return tree.size() - 1;
26     }
27
28     // Inserts x starting from prev_root and returns the new version
29     // root
30     int insert(int prev_root, ll x) {
31         int new_root = add_node(prev_root);
32         int u = new_root, prev_u = prev_root;
33         tree[u].cnt++;
34
35         for (int i = max_bit; i >= 0; i--) {
36             int b = (x >> i) & 1;
37             int prev_child = (prev_u != -1) ? tree[prev_u].nxt[b] : -1;
38             tree[u].nxt[b] = add_node(prev_child);
39             u = tree[u].nxt[b];
40             if (prev_u != -1) prev_u = tree[prev_u].nxt[b];
41             tree[u].cnt++;
42         }
43         roots.pb(new_root);
44         return new_root;
45     }
46
47     // Returns max (x ^ val) over elements inserted in version range [
48     // l_ver, r_ver]
49     ll max_xor(int l_ver, int r_ver, ll x) const {
50         int u_r = roots[r_ver], u_l = (l_ver > 0) ? roots[l_ver - 1] :
51             -1;
52         ll res = 0;
53
54         for (int i = max_bit; i >= 0; i--) {
55             int b = (x >> i) & 1;
56             int want = b ^ 1;
57
58             int count_r = (u_r != -1 && tree[u_r].nxt[want] != -1) ?
59                 tree[tree[u_r].nxt[want]].cnt : 0;
60             int count_l = (u_l != -1 && tree[u_l].nxt[want] != -1) ?
61                 tree[tree[u_l].nxt[want]].cnt : 0;
62
63             if (count_r - count_l > 0) {
64                 res |= (1LL << i);
65                 u_r = tree[u_r].nxt[want];
66                 u_l = (u_l != -1 && tree[u_l].nxt[want] != -1) ? tree[
67                     u_l].nxt[want] : -1;
68             } else {
69                 u_r = (u_r != -1 && tree[u_r].nxt[b] != -1) ? tree[u_r
70                     ].nxt[b] : -1;
71                 u_l = (u_l != -1 && tree[u_l].nxt[b] != -1) ? tree[u_l
72                     ].nxt[b] : -1;
73             }
74         }
75         return res;
76     }
77 };

```

2.5 Disjoint Set Union (DSU)

2.5.1 DSU

```

1 // Disjoint Set Union (DSU) - Path Compression & Union by Size
2 // Time: O(alpha(N)) per operation
3 struct DSU {
4     vector<int> par, sz;
5
6     DSU(int n = 0) : par(n), sz(n, 1) {
7         iota(all(par), 0);
8     }
9
10    int find(int x) {
11        return par[x] == x ? x : par[x] = find(par[x]);
12    }
13
14    bool same(int x, int y) {
15        return find(x) == find(y);
16    }
17
18    bool join(int x, int y) {
19        x = find(x);
20        y = find(y);
21        if (x == y) return false;
22        if (sz[x] < sz[y]) swap(x, y);
23        sz[x] += sz[y];
24        par[y] = x;

```



```

25         return true;
26     }
27
28     int size(int x) {
29         return sz[find(x)];
30     }
31 };

```

2.5.2 DSURollback

```

1 // DSU with Rollback - No path compression to allow backtracking
2 // Time: O(log N) per operation, O(1) rollback
3 struct DSURollback {
4     vector<int> par, sz;
5     vector<pair<int, int>> history; // {child_node, old_parent_size}
6
7     DSURollback(int n = 0) : par(n), sz(n, 1) {
8         iota(all(par), 0);
9     }
10
11     int find(int x) {
12         return par[x] == x ? x : find(par[x]);
13     }
14
15     bool same(int x, int y) {
16         return find(x) == find(y);
17     }
18
19     bool join(int x, int y) {
20         x = find(x);
21         y = find(y);
22         if (x == y) {
23             history.eb(-1, -1);
24             return false;
25         }
26         if (sz[x] < sz[y]) swap(x, y);
27         history.eb(y, sz[x]);
28         sz[x] += sz[y];
29         par[y] = x;
30         return true;
31     }
32
33     int size(int x) {
34         return sz[find(x)];
35     }
36
37     void rollback() {
38         if (history.empty()) return;
39         auto [y, old_sz_x] = history.back();
40         history.pop_back();
41         if (y != -1) {
42             sz[par[y]] = old_sz_x;
43             par[y] = y;
44         }
45     }
46
47     int get_state() const {
48         return history.size();
49     }
50
51     void rollback_to(int state) {
52         while ((int)history.size() > state) rollback();
53     }
54 };

```

2.6 Square Root Decomposition

2.6.1 MoAlgorithm

```

1 class MoArray {
2 private:
3     struct Query {
4         int l, r, idx;
5         int bnd;
6
7         bool operator<(const Query& other) const {
8             if (bnd != other.bnd)
9                 return bnd < other.bnd;
10            return (bnd & 1) ? (r < other.r) : (r > other.r);
11        }
12    };
13
14    int n;
15    int block_size;
16    vector<int> a;
17
18    // --- PROBLEM SPECIFIC STATE ---
19    long long ans;
20    vector<int> freq;
21
22    inline void add(int idx) {
23        if (freq[a[idx]] == 0) ans++;
24        freq[a[idx]]++;
25    }
26
27    inline void remove(int idx) {
28        freq[a[idx]]--;
29        if (freq[a[idx]] == 0) ans--;
30    }
31
32    inline long long get_answer() const {
33        return ans;
34    }
35    // -----
36
37 public:
38     MoArray(const vector<int>& arr) : n(arr.size()), a(arr) {
39         int max_val = 0;
40         for (int x : a) {
41             max_val = max(max_val, x);
42         }
43         freq.assign(max_val + 1, 0);
44         ans = 0;
45     }

```

```

46     vector<long long> solve(const vector<pair<int, int>>& raw_queries)
47     {
48         int q = raw_queries.size();
49         if (q == 0) return {};
50
51         block_size = max(1, (int)(n / sqrt(q)));
52         vector<Query> queries(q);
53
54         for (int i = 0; i < q; ++i) {
55             queries[i].l = raw_queries[i].first;
56             queries[i].r = raw_queries[i].second;
57             queries[i].idx = i;
58             queries[i].bnd = queries[i].l / block_size;
59         }
60
61         sort(queries.begin(), queries.end());
62
63         vector<long long> answers(q);
64         int cur_l = 0, cur_r = -1;
65
66         for (const Query& qry : queries) {
67             while (cur_l > qry.l) add(--cur_l);
68             while (cur_r < qry.r) add(++cur_r);
69             while (cur_l < qry.l) remove(cur_l++);
70             while (cur_r > qry.r) remove(cur_r--);
71             answers[qry.idx] = get_answer();
72         }
73
74         return answers;
75     }
76 };

```

2.6.2 MoOnSubtrees

```

1 class MoSubtree {
2 private:
3     struct Query {
4         int l, r, idx;
5         int bnd;
6
7         bool operator<(const Query& other) const {
8             if (bnd != other.bnd) return bnd < other.bnd;
9             return (bnd & 1) ? (r < other.r) : (r > other.r);
10        }
11    };
12
13    int n;
14    int block_size;
15    vector<vector<int>> adj;
16    vector<int> in, out, flat, clr;
17
18    // --- PROBLEM SPECIFIC STATE ---
19    int mx_freq;
20    vector<int> freq;
21    vector<long long> sum_freq;
22
23    inline void add(int node_idx) {
24        int c = clr[node_idx];
25        sum_freq[freq[c]] -= c;
26        freq[c]++;
27        sum_freq[freq[c]] += c;
28        if (freq[c] > mx_freq) mx_freq = freq[c];
29    }
30
31    inline void remove(int node_idx) {
32        int c = clr[node_idx];
33        sum_freq[freq[c]] -= c;
34        freq[c]--;
35        sum_freq[freq[c]] += c;
36        while (mx_freq > 0 && sum_freq[mx_freq] == 0) mx_freq--;
37    }
38
39    inline long long get_answer() const {
40        return sum_freq[mx_freq];
41    }
42    // -----
43
44    void dfs(int u, int p) {
45        in[u] = flat.size();
46        flat.push_back(u);
47        for (int v : adj[u]) {
48            if (v == p) continue;
49            dfs(v, u);
50        }
51        out[u] = flat.size() - 1;
52    }
53
54 public:
55     MoSubtree(int n, int max_val, const vector<int>& colors)
56         : n(n), adj(n + 1), in(n + 1), out(n + 1), clr(colors) {
57         mx_freq = 0;
58         freq.assign(max_val + 1, 0);
59         sum_freq.assign(n + 1, 0);
60     }
61
62     void add_edge(int u, int v) {
63         adj[u].push_back(v);
64         adj[v].push_back(u);
65     }
66
67     vector<long long> solve(int root, const vector<int>& query_nodes) {
68         flat.reserve(n);
69         dfs(root, 0);
70
71         int q = query_nodes.size();
72         if (q == 0) return {};
73
74         block_size = max(1, (int)(n / sqrt(q)));
75         vector<Query> queries(q);
76
77         for (int i = 0; i < q; ++i) {
78             int u = query_nodes[i];
79             queries[i].l = in[u];

```

```

80     queries[i].r = out[u];
81     queries[i].idx = i;
82     queries[i].bnd = queries[i].l / block_size;
83 }
84
85 sort(queries.begin(), queries.end());
86
87 vector<long long> answers(q);
88 int cur_l = 0, cur_r = -1;
89
90 for (const Query& qry : queries) {
91     while (cur_l > qry.l) add(flat[--cur_l]);
92     while (cur_r < qry.r) add(flat[++cur_r]);
93     while (cur_l < qry.l) remove(flat[cur_l++]);
94     while (cur_r > qry.r) remove(flat[cur_r--]);
95     answers[qry.idx] = get_answer();
96 }
97
98 return answers;
99 }
100 };

```

2.6.3 MoOnPaths

```

1 class MoTreePath {
2 private:
3     int n, LOG, block_size;
4     vector<vector<int>> adj, anc;
5     vector<int> depth, in, out, flat;
6     vector<long long> up, a;
7     vector<bool> exist;
8
9     // --- PROBLEM SPECIFIC STATE ---
10    long long ans;
11    vector<deque<int>> dep; // Tracks nodes by depth
12
13    void add(int idx) {
14        if (!dep[depth[idx]].empty()) {
15            ans += 1ll * a[dep[depth[idx]].back()] * a[idx];
16        }
17        dep[depth[idx]].push_back(idx);
18    }
19
20    void remove(int idx) {
21        if (dep[depth[idx]].size() == 2) {
22            ans -= a[*dep[depth[idx]].begin()] * a[*next(dep[depth[idx]].begin());]
23        }
24        if (dep[depth[idx]].front() == idx) {
25            dep[depth[idx]].pop_front();
26        } else {
27            dep[depth[idx]].pop_back();
28        }
29    }
30
31    void update(int idx) {
32        int node = flat[idx];
33        if (exist[node]) {
34            remove(node);
35        } else {
36            add(node);
37        }
38        exist[node] = !exist[node];
39    }
40
41    inline long long get_answer() const {
42        return ans;
43    }
44    // -----
45
46    void dfs(int u, int p = -1) {
47        if (p != -1) {
48            anc[u][0] = p;
49            up[u] = a[u] * a[u] + up[p];
50        } else {
51            depth[u] = 0;
52            up[u] = a[u] * a[u];
53        }
54        in[u] = flat.size();
55        flat.push_back(u);
56
57        for (int v : adj[u]) {
58            if (v == p) continue;
59            depth[v] = depth[u] + 1;
60            dfs(v, u);
61        }
62
63        out[u] = flat.size();
64        flat.push_back(u); // Push again for Eulerian tour
65    }
66
67    void build_lca() {
68        for (int k = 1; k < LOG; k++) {
69            for (int i = 1; i <= n; i++) {
70                anc[i][k] = anc[anc[i][k - 1]][k - 1];
71            }
72        }
73    }
74
75    int lift(int x, int k) {
76        for (int bit = 0; bit < LOG; bit++) {
77            if ((1 << bit) & k) x = anc[x][bit];
78        }
79        return x;
80    }
81
82    int lca(int u, int v) {
83        if (depth[u] < depth[v]) swap(u, v);
84        u = lift(u, depth[u] - depth[v]);
85        if (u == v) return u;
86        for (int bit = LOG - 1; bit >= 0; bit--) {
87            if (anc[u][bit] != anc[v][bit]) {
88                u = anc[u][bit];
89                v = anc[v][bit];

```

```

90     }
91 }
92 return anc[u][0];
93 }
94
95 public:
96     struct Query {
97         int l, r, idx, x = -1;
98         int bnd;
99
100         bool operator<(const Query& other) const {
101             if (bnd != other.bnd) return bnd < other.bnd;
102             return (bnd & 1) ? (r < other.r) : (r > other.r);
103         }
104     };
105
106     MoTreePath(int n_nodes, const vector<long long>& node_values)
107         : n(n_nodes), a(node_values), LOG(log2(n_nodes) + 2) {
108
109         adj.assign(n + 1, vector<int>());
110         anc.assign(n + 1, vector<int>(LOG, 0));
111         depth.assign(n + 1, 0);
112         in.assign(n + 1, 0);
113         out.assign(n + 1, 0);
114         up.assign(n + 1, 0);
115         exist.assign(n + 1, false);
116         dep.assign(n + 1, deque<int>());
117         ans = 0;
118     }
119
120     void add_edge(int u, int v) {
121         adj[u].push_back(v);
122         adj[v].push_back(u);
123     }
124
125     vector<long long> solve(int root, vector<Query>& queries) {
126         flat.clear();
127         dfs(root);
128         build_lca();
129
130         int q = queries.size();
131         if (q == 0) return {};
132
133         block_size = max(1, (int)(flat.size() / sqrt(q)));
134
135         for (auto& qry : queries) {
136             qry.bnd = qry.l / block_size;
137         }
138
139         sort(queries.begin(), queries.end());
140
141         vector<long long> answers(q);
142         int cur_l = 0, cur_r = -1;
143
144         for (const Query& qry : queries) {
145             while (cur_l < qry.l) update(cur_l++);
146             while (cur_r > qry.r) update(cur_r--);
147             while (cur_l > qry.l) update(--cur_l);
148             while (cur_r < qry.r) update(++cur_r);
149
150             answers[qry.idx] = get_answer() + (qry.x != -1 ? up[qry.x] : 0);
151         }
152         return answers;
153     }
154 };

```

2.7 Balanced Binary Search Tree (BBST)

2.7.1 Treap

```

1 mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
2
3 template <typename T>
4 struct Treap {
5     struct Node {
6         T key, val, sum;
7         T lazy = 0; // 1. Added lazy tag
8         int sz = 1;
9         uint64_t p = rng();
10        Node *l = nullptr, *r = nullptr;
11
12        Node(T k, T v = T()) : key(k), val(v), sum(v) {}
13    } * root = nullptr;
14
15    ~Treap() { destroy(root); }
16
17    void destroy(Node* t) {
18        if (!t) return;
19        destroy(t->l);
20        destroy(t->r);
21        delete t;
22    }
23
24    int sz(Node* t) { return t ? t->sz : 0; }
25    T sub_val(Node* t) { return t ? t->sum : 0; }
26
27    // 2. The Push Function: Propagates pending updates to children
28    void push(Node* t) {
29        if (!t || t->lazy == 0) return;
30
31        if (t->l) {
32            t->l->key += t->lazy; // Shift the key (critical for the DP solution)
33            t->l->val += t->lazy; // Shift the value
34            t->l->sum += t->lazy * sz(t->l); // Update subtree sum
35            t->l->lazy += t->lazy; // Pass the tag down
36        }
37        if (t->r) {
38            t->r->key += t->lazy;
39            t->r->val += t->lazy;
40            t->r->sum += t->lazy * sz(t->r);
41            t->r->lazy += t->lazy;
42        }

```

```

43     t->lazy = 0; // Clear the tag on the current node
44 }
45
46 Node* update(Node* t) {
47     if (t) {
48         t->sz = 1 + sz(t->l) + sz(t->r);
49         t->sum = t->val + sub_val(t->l) + sub_val(t->r);
50     }
51     return t;
52 }
53
54 Node* merge(Node* l, Node* r) {
55     push(l); // 3. Push before making routing decisions
56     push(r);
57     if (!l || !r) return l ? l : r;
58     if (l->p > r->p) {
59         l->r = merge(l->r, r);
60         return update(l);
61     }
62     r->l = merge(l, r->l);
63     return update(r);
64 }
65
66 pair<Node*, Node*> split(Node* t, T k) {
67     if (!t) return {nullptr, nullptr};
68     push(t); // 3. Push before evaluating keys for splits
69     if (t->key <= k) {
70         auto [l, r] = split(t->r, k);
71         t->r = l;
72         return {update(t), r};
73     }
74     auto [l, r] = split(t->l, k);
75     t->l = r;
76     return {l, update(t)};
77 }
78
79 // --- Core Operations ---
80
81 // Range Update Method
82 void add_range(T l_val, T r_val, T add_val) {
83     if (l_val > r_val) return;
84
85     auto [left_mid, right] = split(root, r_val);
86     auto [left, mid] = split(left_mid, l_val - 1);
87
88     // Apply the lazy tag to the entire isolated segment
89     if (mid) {
90         mid->key += add_val;
91         mid->val += add_val;
92         mid->sum += add_val * sz(mid);
93         mid->lazy += add_val;
94     }
95
96     root = merge(merge(left, mid), right);
97 }
98
99 bool search(T x) {
100     Node* curr = root;
101     while (curr) {
102         push(curr); // Push during traversal
103         if (curr->key == x) return true;
104         curr = (x < curr->key) ? curr->l : curr->r;
105     }
106     return false;
107 }
108
109 void insert(T x, T val = T()) {
110     auto [l, r] = split(root, x);
111     auto [l2, r2] = split(l, x - 1);
112     if (!r2) {
113         r2 = new Node(x, val);
114     } else {
115         push(r2); // Push before modifying
116         r2->val += val;
117         update(r2);
118     }
119     root = merge(merge(l2, r2), r);
120 }
121
122 void erase(T x) {
123     auto [l, r] = split(root, x);
124     auto [l2, r2] = split(l, x - 1);
125     destroy(r2);
126     root = merge(l2, r);
127 }
128
129 // --- Utility Operations ---
130
131 Node* find_by_order(Node* t, int k) {
132     if (!t) return nullptr;
133     push(t); // Push before accessing children sizes
134     int left_sz = sz(t->l);
135     if (k < left_sz) return find_by_order(t->l, k);
136     if (k == left_sz) return t;
137     return find_by_order(t->r, k - left_sz - 1);
138 }
139
140 Node* find_by_order(int k) { return find_by_order(root, k); }
141
142 int order_of_key(Node* t, T k) {
143     if (!t) return 0;
144     push(t); // Push before comparing keys
145     if (t->key >= k) return order_of_key(t->l, k);
146     return sz(t->l) + 1 + order_of_key(t->r, k);
147 }
148
149 int order_of_key(T k) { return order_of_key(root, k); }
150
151 void print(Node* t) {
152     if (!t) return;
153     push(t); // Push before printing to get true values
154     print(t->l);
155     cout << t->key << " ";
156     print(t->r);
157 }
158
159 void print() {

```

```

157     print(root);
158     cout << '\n';
159 }
160 };

```

2.7.2 ImplicitTreap

```

1 mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
2
3 template <typename T>
4 struct ImplicitTreap {
5     struct Node {
6         T val, sum;
7         T lazy = 0; // Lazy tag for range additions
8         bool rev = false; // Lazy tag for range reversals
9         int sz = 1;
10        uint64_t p = rng();
11        Node *l = nullptr, *r = nullptr;
12
13        Node(T v = T()) : val(v), sum(v) {}
14    } *root = nullptr;
15
16    // Default Constructor
17    ImplicitTreap() = default;
18
19    // Vector Constructor - Builds treap in O(N) time
20    ImplicitTreap(const vector<T>& a) {
21        build(a);
22    }
23
24    ~ImplicitTreap() { destroy(root); }
25
26    void destroy(Node* t) {
27        if (!t) return;
28        destroy(t->l);
29        destroy(t->r);
30        delete t;
31    }
32
33    int sz(Node* t) { return t ? t->sz : 0; }
34    T sub_val(Node* t) { return t ? t->sum : 0; }
35
36    // Propagates pending lazy updates to children
37    void push(Node* t) {
38        if (!t) return;
39
40        // 1. Process pending reversal
41        if (t->rev) {
42            swap(t->l, t->r);
43            if (t->l) t->l->rev ^= true;
44            if (t->r) t->r->rev ^= true;
45            t->rev = false;
46        }
47
48        // 2. Process pending value addition
49        if (t->lazy != 0) {
50            if (t->l) {
51                t->l->val += t->lazy;
52                t->l->sum += t->lazy * sz(t->l);
53                t->l->lazy += t->lazy;
54            }
55            if (t->r) {
56                t->r->val += t->lazy;
57                t->r->sum += t->lazy * sz(t->r);
58                t->r->lazy += t->lazy;
59            }
60            t->lazy = 0; // Clear the tag
61        }
62    }
63
64    Node* update(Node* t) {
65        if (t) {
66            t->sz = 1 + sz(t->l) + sz(t->r);
67            t->sum = t->val + sub_val(t->l) + sub_val(t->r);
68        }
69        return t;
70    }
71
72    // --- O(N) Linear Time Build ---
73    void build(const vector<T>& a) {
74        destroy(root);
75        root = nullptr;
76        if (a.empty()) return;
77
78        vector<Node*> st;
79        for (const T& val : a) {
80            Node* curr = new Node(val);
81            Node* last = nullptr;
82
83            // Maintain max-heap property on random priority 'p'
84            while (!st.empty() && st.back()->p < curr->p) {
85                last = st.back();
86                st.pop_back();
87            }
88
89            curr->l = last;
90            if (!st.empty()) {
91                st.back()->r = curr;
92            }
93            st.push_back(curr);
94        }
95
96        // Post-order pass to compute subtree sizes and sums in O(N)
97        auto recompute = [&](auto& self, Node* t) -> void {
98            if (!t) return;
99            self(self, t->l);
100            self(self, t->r);
101            update(t);
102        };
103
104        root = st.front(); // Bottom of stack is the root
105        recompute(recompute, root);
106    }
107

```

```

108 // --- Dynamic Array Operations ---
109
110 Node* merge(Node* l, Node* r) {
111     push(l);
112     push(r);
113     if (!l || !r) return l ? l : r;
114     if (l->p > r->p) {
115         l->r = merge(l->r, r);
116         return update(l);
117     }
118     r->l = merge(l, r->l);
119     return update(r);
120 }
121
122 pair<Node*, Node*> split(Node* t, int k) {
123     if (!t) return {nullptr, nullptr};
124     push(t);
125
126     int left_sz = sz(t->l);
127     if (left_sz < k) {
128         auto [l, r] = split(t->r, k - left_sz - 1);
129         t->r = l;
130         return {update(t), r};
131     } else {
132         auto [l, r] = split(t->l, k);
133         t->l = r;
134         return {l, update(t)};
135     }
136 }
137
138 int size() { return sz(root); }
139
140 void insert(int idx, T val) {
141     auto [l, r] = split(root, idx);
142     Node* node = new Node(val);
143     root = merge(merge(l, node), r);
144 }
145
146 void erase(int idx) {
147     if (idx < 0 || idx >= sz(root)) return;
148     auto [left_mid, right] = split(root, idx + 1);
149     auto [left, mid] = split(left_mid, idx);
150     destroy(mid);
151     root = merge(left, right);
152 }
153
154 void reverse_range(int l, int r) {
155     if (l >= r || l < 0 || r >= sz(root)) return;
156
157     auto [left_mid, right] = split(root, r + 1);
158     auto [left, mid] = split(left_mid, l);
159
160     if (mid) mid->rev ^= true;
161
162     root = merge(merge(left, mid), right);
163 }
164
165 void add_range(int l, int r, T add_val) {
166     if (l > r || l < 0 || r >= sz(root)) return;
167
168     auto [left_mid, right] = split(root, r + 1);
169     auto [left, mid] = split(left_mid, l);
170
171     if (mid) {
172         mid->val += add_val;
173         mid->sum += add_val * sz(mid);
174         mid->lazy += add_val;
175     }
176
177     root = merge(merge(left, mid), right);
178 }
179
180 T query_range(int l, int r) {
181     if (l > r || l < 0 || r >= sz(root)) return T();
182
183     auto [left_mid, right] = split(root, r + 1);
184     auto [left, mid] = split(left_mid, l);
185
186     T ans = sub_val(mid);
187
188     root = merge(merge(left, mid), right);
189     return ans;
190 }
191
192 T get(int idx) {
193     return query_range(idx, idx);
194 }
195
196 void print(Node* t) {
197     if (!t) return;
198     push(t);
199     print(t->l);
200     cout << t->val << " ";
201     print(t->r);
202 }
203
204 void print() {
205     print(root);
206     cout << '\n';
207 }
208 };

```

2.8 Policy Based Data Structures

2.8.1 OrderedSet

```

1 // Policy-Based Data Structure: ordered_set
2 // Time: O(log N) per operation
3 #include <ext/pb_ds/assoc_container.hpp>
4 #include <ext/pb_ds/tree_policy.hpp>
5
6 using namespace __gnu_pbds;
7 template<typename T>

```

```

8 using ordered_set = tree<T, null_type, less<T>, rb_tree_tag,
9     tree_order_statistics_node_update>;
10
11 // Usage:
12 // st.find_by_order(k): Iterator to k-th element (0-based)
13 // st.order_of_key(x): Number of elements strictly smaller than x

```

2.9 Monotonic Data Structures

2.9.1 MonotonicStack

```

1 // Monotonic Stack - Time: O(N), Space: O(N)
2 // Computes Next/Previous Smaller and Greater element indices for all
3 // positions
4 template<typename T = ll>
5 struct MonotonicStack {
6     int n;
7     vector<T> a;
8     vector<int> prev_less, next_less;
9     vector<int> prev_greater, next_greater;
10
11     MonotonicStack(const vector<T>& arr) : n(arr.size()), a(arr),
12         prev_less(n, -1), next_less(n, n),
13         prev_greater(n, -1), next_greater(n, n) {
14
15         vector<int> st;
16         // Next Less & Previous Less
17         for (int i = 0; i < n; i++) {
18             while (!st.empty() && a[st.back()] >= a[i]) {
19                 next_less[st.back()] = i;
20                 st.pop_back();
21             }
22             if (!st.empty()) prev_less[i] = st.back();
23             st.pb(i);
24         }
25
26         st.clear();
27         // Next Greater & Previous Greater
28         for (int i = 0; i < n; i++) {
29             while (!st.empty() && a[st.back()] <= a[i]) {
30                 next_greater[st.back()] = i;
31                 st.pop_back();
32             }
33             if (!st.empty()) prev_greater[i] = st.back();
34             st.pb(i);
35         }
36 };

```

2.9.2 TwoStackQueue

```

1 // Monotonic Queue via Two Stacks (Sliding Window Min/Max)
2 // Space: O(N), Time: O(1) amortized per push/pop/get
3 struct TwoStackQ {
4     struct Node {
5         ll mx = -llinf, mn = llinf, val;
6
7         Node() : val(0) {}
8         Node(ll x) : mx(x), mn(x), val(x) {}
9     };
10
11     stack<Node> a, b;
12
13     int size() {
14         return a.size() + b.size();
15     }
16
17     void mrg(Node &a, Node &b) {
18         a.mn = min(a.mn, b.mn);
19         a.mx = max(a.mx, b.mx);
20     }
21
22     // Pushes value into the queue
23     void push(ll val) {
24         auto nd = Node(val);
25         if (!a.empty()) mrg(nd, a.top());
26         a.push(nd);
27     }
28
29     // Transfers elements from stack a to stack b when b is empty
30     void move() {
31         while (!a.empty()) {
32             auto nd = Node(a.top().val);
33             if (!b.empty()) mrg(nd, b.top());
34             b.push(nd);
35             a.pop();
36         }
37     }
38
39     // Returns min/max of all current elements in the queue in O(1)
40     Node get() {
41         Node res;
42         if (!b.empty()) mrg(res, b.top());
43         if (!a.empty()) mrg(res, a.top());
44         return res;
45     }
46
47     // Removes the oldest element from the queue
48     void pop() {
49         if (b.empty()) move();
50         if (!b.empty()) b.pop();
51     }
52 };

```

2.9.3 MonotonicQueue2D

```

1 // 2D Monotonic Queue (Sliding Window 2D Min/Max) - Time: O(N * M),
2 // Space: O(N * M)
3 // Computes min in every K x L subgrid of an N x M matrix
4 template<typename T = ll>
5 struct MonotonicQueue2D {
6     int n, m;

```

```

7 // Returns a matrix res where res[i][j] is the min in subgrid [i..i
  +K-1][j..j+L-1]
8 vector<vector<T>> sliding_window_2d(const vector<vector<T>>& grid,
  int K, int L) {
9     n = grid.size();
10    m = grid[0].size();
11
12    // 1. Sliding window min horizontally along each row
13    vector<vector<T>> row_min(n, vector<T>(m - L + 1));
14    for (int i = 0; i < n; i++) {
15        deque<int> dq;
16        for (int j = 0; j < m; j++) {
17            while (!dq.empty() && grid[i][dq.back()] >= grid[i][j])
18                dq.pop_back();
19            dq.pb(j);
20            if (dq.front() <= j - L) dq.pop_front();
21            if (j >= L - 1) row_min[i][j - L + 1] = grid[i][dq.
22                front()];
23        }
24    }
25
26    // 2. Sliding window min vertically down each column of row_min
27    vector<vector<T>> res(n - K + 1, vector<T>(m - L + 1));
28    for (int j = 0; j < m - L + 1; j++) {
29        deque<int> dq;
30        for (int i = 0; i < n; i++) {
31            while (!dq.empty() && row_min[dq.back()][j] >= row_min[
32                i][j]) dq.pop_back();
33            dq.pb(i);
34            if (dq.front() <= i - K) dq.pop_front();
35            if (i >= K - 1) res[i - K + 1][j] = row_min[dq.front()
36                ][j];
37        }
38    }
39    return res;
40 }
41 };

```

3 Graph Theory

3.1 Graph Traversal

3.1.1 TopoSort

```

1 // Topological Sort (Kahn's Algorithm) - O(V + E)
2 struct TopoSort {
3     int n;
4     vector<vector<int>> adj;
5     vector<int> deg, order;
6
7     TopoSort(int n = 0) : n(n), adj(n + 1), deg(n + 1, 0) {}
8
9     void add_edge(int u, int v) {
10         adj[u].pb(v);
11         deg[v]++;
12     }
13
14     bool solve() {
15         queue<int> q;
16         for (int i = 1; i <= n; i++) {
17             if (!deg[i]) q.push(i);
18         }
19         while (!q.empty()) {
20             int u = q.front();
21             q.pop();
22             order.pb(u);
23             for (int v : adj[u]) {
24                 if (--deg[v] == 0) q.push(v);
25             }
26         }
27         return (int)order.size() == n; // false if graph has cycles
28     }
29 };

```

3.2 Shortest Paths

3.2.1 Dijkstra

```

1 // Dijkstra's Shortest Path Algorithm - O((V + E) log V)
2 struct Dijkstra {
3     int n;
4     vector<vector<pair<int, ll>>> adj;
5     vector<ll> dist;
6     vector<int> par;
7
8     Dijkstra(int n = 0) : n(n), adj(n + 1), dist(n + 1, llinf), par(n +
  1, -1) {}
9
10    void add_edge(int u, int v, ll w, bool directed = false) {
11        adj[u].eb(v, w);
12        if (!directed) adj[v].eb(u, w);
13    }
14
15    void solve(int src) {
16        fill(all(dist), llinf);
17        fill(all(par), -1);
18        priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<>>
  pq;
19
20        dist[src] = 0;
21        pq.emplace(0, src);
22
23        while (!pq.empty()) {
24            auto [d, u] = pq.top();
25            pq.pop();
26            if (d > dist[u]) continue;
27
28            for (auto& [v, w] : adj[u]) {
29                if (dist[u] + w < dist[v]) {
30                    dist[v] = dist[u] + w;
31                    par[v] = u;
32                    pq.emplace(dist[v], v);
33                }
34            }
35        }
36    }
37 };

```

```

33     }
34 }
35 }
36 }
37 };

```

3.2.2 FloydWarshall

```

1 // Floyd-Warshall All-Pairs Shortest Path - O(V^3)
2 struct FloydWarshall {
3     int n;
4     vector<vector<ll>> dist;
5
6     FloydWarshall(int n = 0) : n(n), dist(n + 1, vector<ll>(n + 1,
  llinf)) {}
7     for (int i = 0; i <= n; i++) dist[i][i] = 0;
8 }
9
10    void add_edge(int u, int v, ll w, bool directed = false) {
11        dist[u][v] = min(dist[u][v], w);
12        if (!directed) dist[v][u] = min(dist[v][u], w);
13    }
14
15    void solve() {
16        for (int k = 1; k <= n; k++) {
17            for (int i = 1; i <= n; i++) {
18                for (int j = 1; j <= n; j++) {
19                    if (dist[i][k] < llinf && dist[k][j] < llinf) {
20                        dist[i][j] = min(dist[i][j], dist[i][k] + dist[
  k][j]);
21                    }
22                }
23            }
24        }
25    }
26 };

```

3.2.3 BellmanFord

```

1 // Bellman-Ford Shortest Path with Negative Cycle Detection - O(V * E)
2 struct Edge {
3     int u, v;
4     ll w;
5 };
6
7 struct BellmanFord {
8     int n;
9     vector<Edge> edges;
10    vector<ll> dist;
11    vector<int> par;
12
13    BellmanFord(int n = 0) : n(n), dist(n + 1, llinf), par(n + 1, -1)
  {}
14
15    void add_edge(int u, int v, ll w) {
16        edges.pb({u, v, w});
17    }
18
19    // Returns false if a negative cycle is reachable from src
20    bool solve(int src) {
21        fill(all(dist), llinf);
22        fill(all(par), -1);
23        dist[src] = 0;
24
25        for (int i = 0; i < n - 1; i++) {
26            bool relaxed = false;
27            for (const auto& e : edges) {
28                if (dist[e.u] < llinf && dist[e.u] + e.w < dist[e.v]) {
29                    dist[e.v] = dist[e.u] + e.w;
30                    par[e.v] = e.u;
31                    relaxed = true;
32                }
33            }
34            if (!relaxed) break;
35        }
36
37        for (const auto& e : edges) {
38            if (dist[e.u] < llinf && dist[e.u] + e.w < dist[e.v]) {
39                return false; // Negative cycle detected
40            }
41        }
42        return true;
43    }
44 };

```

3.3 Graph Connectivity

3.3.1 Bridges

```

1 // Bridges (Cut Edges) in Undirected Graph - O(V + E)
2 struct Bridges {
3     int n, timer;
4     vector<vector<int>> adj;
5     vector<int> tin, low;
6     vector<pair<int, int>> bridges;
7
8     Bridges(int n = 0) : n(n), timer(0), adj(n + 1), tin(n + 1, 0), low
  (n + 1, 0) {}
9
10    void add_edge(int u, int v) {
11        adj[u].pb(v);
12        adj[v].pb(u);
13    }
14
15    void dfs(int u, int p = 0) {
16        tin[u] = low[u] = ++timer;
17        for (int v : adj[u]) {
18            if (v == p) continue;
19            if (tin[v]) {
20                low[u] = min(low[u], tin[v]);
21            } else {
22                dfs(v, u);
23                low[u] = min(low[u], low[v]);
24            }
25        }
26    }
27 };

```

```

24         if (low[v] > tin[u]) bridges.pb(u, v);
25     }
26 }
27 }
28
29 void solve() {
30     timer = 0;
31     fill(all(tin), 0);
32     bridges.clear();
33     for (int i = 1; i <= n; i++) {
34         if (!tin[i]) dfs(i);
35     }
36 }
37 };

```

3.3.2 CutPoints

```

1 // Articulation Points (Cut Vertices) in Undirected Graph -  $O(V + E)$ 
2 struct ArticulationPoints {
3     int n, timer;
4     vector<vector<int>> adj;
5     vector<int> tin, low;
6     vector<bool> is_cut;
7
8     ArticulationPoints(int n = 0) : n(n), timer(0), adj(n + 1), tin(n + 1, 0), low(n + 1, 0), is_cut(n + 1, false) {}
9
10    void add_edge(int u, int v) {
11        adj[u].pb(v);
12        adj[v].pb(u);
13    }
14
15    void dfs(int u, int p = 0) {
16        tin[u] = low[u] = ++timer;
17        int children = 0;
18        for (int v : adj[u]) {
19            if (v == p) continue;
20            if (tin[v]) {
21                low[u] = min(low[u], tin[v]);
22            } else {
23                dfs(v, u);
24                low[u] = min(low[u], low[v]);
25                if (low[v] >= tin[u] && p != 0) is_cut[u] = true;
26                children++;
27            }
28        }
29        if (p == 0 && children > 1) is_cut[u] = true;
30    }
31
32    void solve() {
33        timer = 0;
34        fill(all(tin), 0);
35        fill(all(is_cut), false);
36        for (int i = 1; i <= n; i++) {
37            if (!tin[i]) dfs(i);
38        }
39    }
40 };

```

3.3.3 SCC

```

1 // Strongly Connected Components (Tarjan's Algorithm) -  $O(V + E)$ 
2 struct SCC {
3     int n, timer = 0, num_comps = 0;
4     vector<vector<int>> adj;
5     vector<int> tin, low, comp;
6     vector<bool> in_st;
7     stack<int> st;
8     vector<vector<int>> comps;
9
10    SCC(int n = 0) : n(n), adj(n + 1), tin(n + 1, 0), low(n + 1, 0), comp(n + 1, -1), in_st(n + 1, false) {}
11
12    void add_edge(int u, int v) {
13        adj[u].pb(v);
14    }
15
16    void dfs(int u) {
17        tin[u] = low[u] = ++timer;
18        st.push(u);
19        in_st[u] = true;
20
21        for (int v : adj[u]) {
22            if (!tin[v]) {
23                dfs(v);
24                low[u] = min(low[u], low[v]);
25            } else if (in_st[v]) {
26                low[u] = min(low[u], tin[v]);
27            }
28        }
29
30        if (low[u] == tin[u]) {
31            comps.pb({});
32            while (true) {
33                int v = st.top();
34                st.pop();
35                in_st[v] = false;
36                comp[v] = num_comps;
37                comps.back().pb(v);
38                if (u == v) break;
39            }
40            num_comps++;
41        }
42    }
43
44    void solve() {
45        for (int i = 1; i <= n; i++) {
46            if (!tin[i]) dfs(i);
47        }
48    }
49
50    vector<vector<int>> build_dag() {
51        vector<vector<int>> dag(num_comps);
52        for (int u = 1; u <= n; u++) {

```

```

53         for (int v : adj[u]) {
54             if (comp[u] != comp[v]) dag[comp[u]].pb(comp[v]);
55         }
56     }
57     return dag;
58 }
59 };

```

3.4 Satisfiability

3.4.1 2SAT

```

1 struct TwoSat {
2     int N;
3     vector<pair<int, int>> edges;
4
5     TwoSat(int _N = 0) {
6         init(_N);
7     }
8
9     void init(int _N = 0) {
10         N = _N;
11     }
12
13     int addVar() { return N++; }
14
15     //  $x$  or  $y$ , edges will be refined in the end
16     void either(int x, int y) {
17         x = max(2 * x, -1 - 2 * x);
18         y = max(2 * y, -1 - 2 * y);
19         edges.pb({x, y});
20     }
21
22     void implies(int x, int y) {
23         either(-x, y);
24     }
25
26     void must(int x) {
27         either(x, x);
28     }
29
30     void XOR(int x, int y) {
31         either(x, y);
32         either(-x, -y);
33     }
34
35     vector<bool> solve(int _N = -1) {
36         if (_N != -1) N = _N;
37         SCC scc;
38         scc.init(2 * N);
39         for (auto e : edges) {
40             scc.addEdge(e.st ^ 1, e.nd);
41             scc.addEdge(e.nd ^ 1, e.st);
42         }
43         scc.gen();
44         for (int i = 0; i < 2 * N; ++i) {
45             if (scc.compOf[i] == scc.compOf[i ^ 1]) return {};
46         }
47         vector<vector<int>> comps = scc.comps;
48         reverse(all(comps));
49         vector<int> compOf(2 * N);
50         for (int i = 0; i < comps.size(); ++i) {
51             for (auto e : comps[i])
52                 compOf[e] = i;
53         }
54         vector<int> tmp(comps.size());
55         for (int i = 0; i < comps.size(); ++i) {
56             if (!tmp[i]) {
57                 tmp[i] = 1;
58                 for (auto e : comps[i])
59                     tmp[compOf[e ^ 1]] = -1;
60             }
61         }
62         vector<bool> ans(N);
63         for (int i = 0; i < N; ++i)
64             ans[i] = tmp[compOf[2 * i]] == 1;
65         return ans;
66     }
67
68     void atMostOne(const vector<int> &li) {
69         if (li.size() <= 1) return;
70         int last = -li[1];
71         for (int i = 2; i < li.size(); i++) {
72             int next = addVar();
73             implies(li[i], last);
74             either(last, next);
75             implies(li[i], next);
76             last = -next;
77         }
78         implies(li[0], last);
79     }
80 };

```

3.5 Eulerian Path

3.5.1 EulerianPath

```

1 // Hierholzer's Algorithm for Eulerian Path / Circuit -  $O(V + E)$ 
2 // Works for both Directed and Undirected graphs
3 struct EulerianPath {
4     int n, m;
5     vector<vector<pair<int, int>>> adj; // {neighbor, edge_id}
6     vector<bool> used_edge;
7     vector<int> in_deg, out_deg, path;
8
9     EulerianPath(int n = 0, int m = 0) : n(n), m(m), adj(n + 1), used_edge(m, false), in_deg(n + 1, 0), out_deg(n + 1, 0) {}
10
11    void add_edge(int u, int v, int edge_id, bool directed = true) {
12        adj[u].pb({v, edge_id});
13        out_deg[u]++;
14        in_deg[v]++;
15        if (!directed) {
16            adj[v].pb({u, edge_id});

```



```

17         out_deg[v]++;
18         in_deg[u]++;
19     }
20 }
21
22 // Finds Eulerian path starting at start_node. Returns false if
23 // graph is not Eulerian
24 bool solve(int start_node) {
25     vector<int> ptr(n + 1, 0);
26     stack<int> st;
27     st.push(start_node);
28
29     while (!st.empty()) {
30         int u = st.top();
31         if (ptr[u] < adj[u].size()) {
32             auto [v, id] = adj[u][ptr[u]++];
33             if (!used_edge[id]) {
34                 used_edge[id] = true;
35                 st.push(v);
36             }
37         } else {
38             path.pb(u);
39             st.pop();
40         }
41     }
42     reverse(all(path));
43
44     // Verify that every edge was traversed
45     for (bool used : used_edge) {
46         if (!used) return false;
47     }
48     return true;
49 };

```

3.6 Flow and Min Cut

3.6.1 Dinic

```

1 struct Dinic {
2     struct Edge {
3         int to, rev;
4         ll c, oc;
5         ll flow() { return max(oc - c, 0LL); } // if you need flows
6     };
7
8     vector<int> lvl, ptr, q;
9     vector<vector<Edge>> > adj;
10
11     Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {}
12
13     void addEdge(int a, int b, ll c, ll rcap = 0) {
14         adj[a].push_back({b, (int) adj[b].size(), c, c});
15         adj[b].push_back({a, (int) adj[a].size() - 1, rcap, rcap});
16     }
17
18     ll dfs(int v, int t, ll f) {
19         if (v == t || !f) return f;
20         for (int &i = ptr[v]; i < (int) adj[v].size(); i++) {
21             Edge &e = adj[v][i];
22             if (lvl[e.to] == lvl[v] + 1)
23                 if (ll p = dfs(e.to, t, min(f, e.c))) {
24                     e.c -= p, adj[e.to][e.rev].c += p;
25                     return p;
26                 }
27         }
28         return 0;
29     }
30
31     ll calc(int s, int t) {
32         ll flow = 0;
33         q[0] = s;
34         for (int L = 0; L < 31; L++)
35             do {
36                 // 'int L=30' maybe faster for random data
37                 lvl = ptr = vector<int>((int) q.size());
38                 int qi = 0, qe = lvl[s] = 1;
39                 while (qi < qe && !lvl[t]) {
40                     int v = q[qi++];
41                     for (Edge e : adj[v])
42                         if (!lvl[e.to] && e.c >> (30 - L))
43                             q[qe++] = e.to, lvl[e.to] = lvl[v] + 1;
44                 }
45                 while (ll p = dfs(s, t, LLONG_MAX)) flow += p;
46             } while (lvl[t]);
47         return flow;
48     }
49
50     bool leftOfMinCut(int a) { return lvl[a] != 0; }
51 };

```

3.6.2 MCMF

```

1 struct Edge {
2     int to;
3     int cost;
4     int cap, flow, backEdge;
5 };
6
7 struct MCMF
8 {
9
10     const int inf = 1000000010;
11     int n;
12     vector<vector<Edge>> > g;
13
14     MCMF(int _n) {
15         n = _n + 1;
16         g.resize(n);
17     }
18
19     void addEdge(int u, int v, int cap, int cost) {
20         Edge e1 = {v, cost, cap, 0, (int) g[v].size()};

```

```

21         Edge e2 = {u, -cost, 0, 0, (int) g[u].size()};
22         g[u].push_back(e1);
23         g[v].push_back(e2);
24     }
25
26     pair<int, int> minCostMaxFlow(int s, int t) {
27         int flow = 0;
28         int cost = 0;
29         vector<int> state(n), from(n), from_edge(n);
30         vector<int> d(n);
31         deque<int> q;
32         while (true) {
33             for (int i = 0; i < n; i++)
34                 state[i] = 2, d[i] = inf, from[i] = -1;
35             state[s] = 1;
36             q.clear();
37             q.push_back(s);
38             d[s] = 0;
39             while (!q.empty()) {
40                 int v = q.front();
41                 q.pop_front();
42                 state[v] = 0;
43                 for (int i = 0; i < (int) g[v].size(); i++) {
44                     Edge e = g[v][i];
45                     if (e.flow >= e.cap || (d[e.to] <= d[v] + e.cost))
46                         continue;
47                     int to = e.to;
48                     d[to] = d[v] + e.cost;
49                     from[to] = v;
50                     from_edge[to] = i;
51                     if (state[to] == 1) continue;
52                     if (!state[to] || (!q.empty() && d[q.front()] > d[to]))
53                         q.push_front(to);
54                     else q.push_back(to);
55                     state[to] = 1;
56                 }
57             }
58             if (d[t] == inf) break;
59             int it = t, addflow = inf;
60             while (it != s) {
61                 addflow = min(addflow,
62                             g[from[it]][from_edge[it]].cap
63                             - g[from[it]][from_edge[it]].flow);
64                 it = from[it];
65             }
66             it = t;
67             while (it != s) {
68                 g[from[it]][from_edge[it]].flow += addflow;
69                 g[it][g[from[it]][from_edge[it]].backEdge].flow -=
70                     addflow;
71                 cost += g[from[it]][from_edge[it]].cost * addflow;
72                 it = from[it];
73             }
74             flow += addflow;
75             return {cost, flow};
76         }
77     };

```

3.7 Matching

3.7.1 HopcroftKarp

```

1 struct HopcroftKarp {
2     vector<int> leftMatch, rightMatch, dist, cur;
3     vector<vector<int>> > a;
4     int n, m;
5
6     HopcroftKarp() {}
7
8     HopcroftKarp(int n, int m) {
9         this->n = n;
10        this->m = m;
11        a = vector<vector<int>> >(n);
12        leftMatch = vector<int>(m, -1);
13        rightMatch = vector<int>(n, -1);
14        dist = vector<int>(n, -1);
15        cur = vector<int>(n, -1);
16    }
17
18    void addEdge(int x, int y) {
19        a[x].push_back(y);
20    }
21
22    int bfs() {
23        int found = 0;
24        queue<int> q;
25        for (int i = 0; i < n; i++)
26            if (rightMatch[i] < 0) dist[i] = 0, q.push(i);
27            else dist[i] = -1;
28
29        while (!q.empty()) {
30            int x = q.front();
31            q.pop();
32            for (int i = 0; i < (int) a[x].size(); i++) {
33                int y = a[x][i];
34                if (leftMatch[y] < 0) found = 1;
35                else if (dist[leftMatch[y]] < 0)
36                    dist[leftMatch[y]] = dist[x] + 1, q.push(leftMatch[
37                        y]);
38            }
39        }
40        return found;
41    }
42
43    int dfs(int x) {
44        for (; cur[x] < (int) a[x].size(); cur[x]++) {
45            int y = a[x][cur[x]];
46            if (leftMatch[y] < 0 || (dist[leftMatch[y]] == dist[x] + 1
47                && dfs(leftMatch[y]))) {
48                leftMatch[y] = x;
49                rightMatch[x] = y;

```

```

49         return 1;
50     }
51 }
52 return 0;
53 }
54
55 int maxMatching() {
56     int match = 0;
57     while (bfs()) {
58         for (int i = 0; i < n; i++) cur[i] = 0;
59         for (int i = 0; i < n; i++)
60             if (rightMatch[i] < 0) match += dfs(i);
61     }
62     return match;
63 }
64 };

```

4 Number Theory

4.1 Euclidean Algorithm

4.1.1 ExtendedEuclid

```

1 long long extGCD(long long a, long long b, long long &x, long long &y)
2 {
3     if (b == 0) {
4         x = 1; y = 0;
5         return a;
6     }
7     long long x1, y1;
8     long long d = extGCD(b, a % b, x1, y1);
9     x = y1;
10    y = x1 - y1 * (a / b);
11    return d;
12 }
13
14 long long modInverse(long long a, long long m) {
15     long long x, y;
16     long long g = extGCD(a, m, x, y);
17     if (g != 1) return -1; // Inverse doesn't exist
18     return (x % m + m) % m;
19 }

```

4.1.2 LinearDiophantine

```

1 // Linear Diophantine Equations (2 Variables & N Variables)
2 // 1. Solves a * x + b * y = c (Particular solution & GCD check)
3 // 2. Solves a_1*x_1 + a_2*x_2 + ... + a_n*x_n = c for N variables
4 struct LinearDiophantine {
5     static ll ext_gcd(ll a, ll b, ll &x, ll &y) {
6         if (b == 0) {
7             x = 1; y = 0;
8             return a;
9         }
10        ll x1, y1;
11        ll d = ext_gcd(b, a % b, x1, y1);
12        x = y1;
13        y = x1 - y1 * (a / b);
14        return d;
15    }
16
17    // Solves a * x + b * y = c. Returns false if no solution exists
18    static bool solve_2var(ll a, ll b, ll c, ll &x, ll &y, ll &g) {
19        if (a == 0 && b == 0) {
20            if (c == 0) { x = y = g = 0; return true; }
21            return false;
22        }
23        g = ext_gcd(abs(a), abs(b), x, y);
24        if (c % g != 0) return false;
25        x *= c / g;
26        y *= c / g;
27        if (a < 0) x = -x;
28        if (b < 0) y = -y;
29        return true;
30    }
31
32    // Solves a_1*x_1 + a_2*x_2 + ... + a_n*x_n = c for N variables.
33    // Returns true if a solution exists and populates x vector
34    static bool solve_nvar(const vector<ll>& a, ll c, vector<ll>& x) {
35        int n = a.size();
36        if (n == 0) return c == 0;
37        x.assign(n, 0);
38
39        vector<ll> prefix_g(n);
40        prefix_g[0] = abs(a[0]);
41        for (int i = 1; i < n; i++) prefix_g[i] = std::gcd(prefix_g[i - 1], a[i]);
42
43        if (prefix_g.back() == 0) return c == 0;
44        if (c % prefix_g.back() != 0) return false;
45
46        ll cur_c = c;
47        for (int i = n - 1; i > 0; i--) {
48            ll x_prev, y_curr, g;
49            solve_2var(prefix_g[i - 1], a[i], cur_c, x_prev, y_curr, g);
50            x[i] = y_curr;
51            cur_c = prefix_g[i - 1] * x_prev;
52        }
53        x[0] = cur_c / a[0];
54        return true;
55    }
56 };

```

4.2 Primes and Divisors

4.2.1 Sieve

```

1 // Linear Sieve & Smallest Prime Factor (SPF) - Time: O(N), Space: O(N)
2 // Computes primes list, SPF array, primality test, and O(log N) prime factorization
3 struct Sieve {
4     int n;

```

```

5     vector<int> primes;
6     vector<int> spf;
7
8     Sieve(int n = 1e7) : n(n), spf(n + 1, 0) {
9         for (int i = 2; i <= n; i++) {
10            if (spf[i] == 0) {
11                spf[i] = i;
12                primes.pb(i);
13            }
14            for (int p : primes) {
15                if (p > spf[i] || (ll)i * p > n) break;
16                spf[i * p] = p;
17            }
18        }
19    }
20
21    bool is_prime(int x) const {
22        return x >= 2 && x <= n && spf[x] == x;
23    }
24
25    // O(log N) prime factorization using precomputed SPF array
26    vector<int> get_prime_factors(int x) const {
27        vector<int> factors;
28        while (x > 1) {
29            factors.pb(spf[x]);
30            x /= spf[x];
31        }
32        return factors;
33    }
34 };

```

4.2.2 Factorization

```

1 // Trial Division Prime Factorization - Time: O(sqrt(N))
2 template <typename T = ll>
3 struct Factorization {
4     static vector<T> get_prime_factors(T n) {
5         vector<T> factors;
6         for (T i = 2; i * i <= n; i++) {
7             while (n % i == 0) {
8                 factors.pb(i);
9                 n /= i;
10            }
11        }
12        if (n > 1) factors.pb(n);
13        return factors;
14    }
15 };

```

4.2.3 Divisors

```

1 // Trial Division Divisor Generation - Time: O(sqrt(N))
2 template <typename T = ll>
3 struct Divisors {
4     static vector<T> get_divisors(T n) {
5         vector<T> divs;
6         for (T i = 1; i * i <= n; i++) {
7             if (n % i == 0) {
8                 divs.pb(i);
9                 if (n / i != i) divs.pb(n / i);
10            }
11        }
12        return divs;
13    }
14 };

```

4.2.4 Sieve1e9

```

1 // Fast Bitset Sieve up to N = 10^9 - Time: ~0.3s for 10^9, Space: N / 16 bytes (~60MB)
2 // Stores odd numbers only (index i represents number 2*i + 1)
3 template <int MAXPR = (int)1e9>
4 struct Sieve1e9 {
5     bitset<MAXPR / 2> is_prime_odd;
6
7     Sieve1e9(int n = MAXPR) {
8         is_prime_odd.set();
9         is_prime_odd[0] = 0; // 1 is not prime
10        for (int i = 1; (2 * i + 1) * (2 * i + 1) <= n; i++) {
11            if (is_prime_odd[i]) {
12                int p = 2 * i + 1;
13                for (int j = 2 * i * (i + 1); j <= n / 2; j += p) {
14                    is_prime_odd[j] = 0;
15                }
16            }
17        }
18    }
19
20    bool is_prime(int x) const {
21        if (x == 2) return true;
22        if (x < 2 || x % 2 == 0) return false;
23        return is_prime_odd[x / 2];
24    }
25 };

```

4.2.5 SegmentedSieve

```

1 // Segmented Sieve for Range [L, R] - Time: O(sqrt(R)) + (R - L) log log R, Space: O(sqrt(R)) + R - L
2 // Computes all prime numbers in range [L, R] where R <= 1e12 and (R - L) <= 1e7
3 struct SegmentedSieve {
4     static vector<ll> get_primes_in_range(ll L, ll R) {
5         ll limit = sqrt(R);
6         vector<bool> is_prime_small(limit + 1, true);
7         vector<ll> primes;
8         for (ll i = 2; i <= limit; i++) {
9             if (is_prime_small[i]) {
10                primes.pb(i);
11                for (ll j = i * i; j <= limit; j += i) is_prime_small[j] = false;
12            }
13        }

```

```

14     vector<bool> is_prime_range(R - L + 1, true);
15     for (ll p : primes) {
16         ll start = max(p * p, (L + p - 1) / p * p);
17         for (ll j = start; j <= R; j += p) {
18             is_prime_range[j - L] = false;
19         }
20     }
21     if (L == 1) is_prime_range[0] = false;
22
23     vector<ll> result;
24     for (ll i = 0; i <= R - L; i++) {
25         if (is_prime_range[i]) result.pb(L + i);
26     }
27     return result;
28 }
29
30 };

```

4.3 Modular Arithmetic

4.3.1 CRT

```

1 long long extGCD(long long a, long long b, long long &x, long long &y)
2 {
3     if (b == 0) {
4         x = 1; y = 0;
5         return a;
6     }
7     long long x1, y1;
8     long long d = extGCD(b, a % b, x1, y1);
9     x = y1;
10    y = x1 - y1 * (a / b);
11    return d;
12 }
13
14 // Solves  $x = a_i \pmod{m_i}$ . Returns  $\{a, lcm(m_1, \dots, m_k)\}$ .
15 pair<long long, long long> CRT(const vector<long long>&a, const vector
16 <long long>&m) {
17     long long res = a[0], lcm = m[0];
18     for (int i = 1; i < a.size(); i++) {
19         long long x, y;
20         long long g = extGCD(lcm, m[i], x, y);
21         if ((a[i] - res) % g != 0) return {-1, -1}; // No solution
22
23         long long step = m[i] / g;
24         long long k = ((a[i] - res) / g) % step * (x % step) % step;
25         res += k * lcm;
26         lcm *= step;
27         res = (res % lcm + lcm) % lcm;
28     }
29     return {res, lcm};
30 }

```

4.3.2 FloorSum

```

1 // Floor Sum - Time:  $O(\log m)$ , Space:  $O(\log m)$ 
2 // Computes  $\sum_{i=0}^{n-1} \text{floor}((a * i + b) / m)$ 
3 // Valid for  $0 \leq n, 0 < m, 0 \leq a, 0 \leq b$ 
4 ll floor_sum(ll n, ll m, ll a, ll b) {
5     ll ans = 0;
6     if (a >= m) {
7         ans += (n - 1) * n / 2 * (a / m);
8         a %= m;
9     }
10    if (b >= m) {
11        ans += n * (b / m);
12        b %= m;
13    }
14
15    ll y_max = (a * n + b) / m;
16    ll x_max = y_max * m - b;
17    if (y_max == 0) return ans;
18    ans += (n - (x_max + a - 1) / a) * y_max;
19    ans += floor_sum(y_max, a, m, (a - x_max % a) % a);
20    return ans;
21 }

```

5 Combinatorics

5.1 StarsAndBars

```

1 // Stars and Bars (Number of Solutions to  $x_1 + x_2 + \dots + x_k = n$ ) -
2 //  $O(1)$ 
3 // 1. Non-negative integer solutions ( $x_i \geq 0$ ):  $C(n + k - 1, k - 1)$ 
4 // 2. Positive integer solutions ( $x_i \geq 1$ ):  $C(n - 1, k - 1)$ 
5 struct StarsAndBars {
6     // Requires precomputed Combination  $C(n, k)$ 
7     static ll count_non_negative(ll n, ll k, function<ll(ll, ll)> nCr)
8     {
9         if (n < 0 || k <= 0) return 0;
10        return nCr(n + k - 1, k - 1);
11    }
12
13    static ll count_positive(ll n, ll k, function<ll(ll, ll)> nCr) {
14        if (n < k || k <= 0) return 0;
15        return nCr(n - 1, k - 1);
16    }
17 }

```

5.2 BurnsideLemma

```

1 // Burnside's Lemma & óPlya Enumeration Theorem - Time:  $O(N)$ 
2 // Counts distinct colorings under symmetry group G
3 // Formula: Number of orbits =  $(1 / |G|) * \sum_{\{g \text{ in } G\}} (k^{\text{number\_of\_cycles}(g)})$ 
4 struct BurnsideLemma {
5     // Calculates number of distinct necklets of length n with k colors
6     // under rotations
7     static ll necklace(int n, int k) {
8         ll total = 0;
9         for (int i = 0; i < n; i++) {
10             total += fast_pow(k, std::gcd(i, n));
11         }
12     }
13 }

```

```

10     }
11     return total / n;
12 }
13
14 // Calculates number of distinct bracelets of length n with k
15 // colors under rotation + reflection
16 static ll bracelet(int n, int k) {
17     ll total = 0;
18     // Rotations
19     for (int i = 0; i < n; i++) {
20         total += fast_pow(k, std::gcd(i, n));
21     }
22     // Reflections
23     if (n & 1) {
24         total += (ll)n * fast_pow(k, (n + 1) / 2);
25     } else {
26         total += (ll)(n / 2) * fast_pow(k, n / 2) + (ll)(n / 2) *
27             fast_pow(k, n / 2 + 1);
28     }
29     return total / (2 * n);
30 }
31
32 private:
33 static ll fast_pow(ll base, ll exp) {
34     ll res = 1;
35     while (exp > 0) {
36         if (exp & 1) res *= base;
37         base *= base;
38         exp >>= 1;
39     }
40     return res;
41 }

```

6 Math

6.1 Modular Arithmetic

6.1.1 ModularArithmetic

```

1 int fact[N], inv[N], invFact[N]; //used in nCr(), nPr(), calcFact()
2
3 int modSum(ll a, ll b) {
4     if (a < 0)
5         a += MOD;
6     if (b < 0)
7         b += MOD;
8     a += b;
9     if (a >= MOD)
10        a -= MOD;
11    return a;
12
13    a = (a % MOD + MOD) % MOD;
14    b = (b % MOD + MOD) % MOD;
15    return (a + b) % MOD;
16 }
17
18 int modProd(ll a, ll b) {
19     if (a < 0)
20         a += MOD;
21     if (b < 0)
22         b += MOD;
23     a *= b;
24     if (a >= MOD)
25         a %= MOD;
26    return a;
27
28    a = (a % MOD + MOD) % MOD;
29    b = (b % MOD + MOD) % MOD;
30    return (a * b) % MOD;
31 }
32
33 int modPow(int a, ll b) {
34     int result = 1;
35     while (b) {
36         if (b & 1)
37             result = modProd(result, a);
38         a = modProd(a, a);
39         b >>= 1;
40     }
41    return result;
42 }
43
44 int modInverse(int a) {
45     return modPow(a, MOD - 2);
46 }
47
48 int modDiv(int a, int b) {
49     return modProd(a, modInverse(b));
50 }
51
52 int nCr(int n, int r) {
53     if (r < 0 || r > n)
54         return 0;
55     // return modDiv(fact[n], modProd(fact[r], fact[n - r]));
56     return modProd(fact[n], modProd(invFact[r], invFact[n - r]));
57 }
58
59 void build() {
60     fact[0] = invFact[0] = 1;
61     for (int i = 1; i < N; i++) {
62         fact[i] = modProd(fact[i - 1], i);
63         invFact[i] = modDiv(invFact[i - 1], i);
64         inv[i] = modInverse(i);
65     }
66 }

```

```

75     }
76 }

```

6.1.2 ModularIntClass

```

1 struct Mint {
2     int v;
3
4     Mint(long long val = 0) {
5         v = int(val % MOD);
6         if (v < 0) v += MOD;
7     }
8
9     Mint operator+(const Mint& o) const { return Mint(v + o.v); }
10    Mint operator-(const Mint& o) const { return Mint(v - o.v); }
11    Mint operator*(const Mint& o) const { return Mint(1LL * v * o.v); }
12    Mint operator/(const Mint& o) const { return *this * o.inv(); }
13
14    Mint& operator+=(const Mint& o) {
15        v += o.v;
16        if (v >= MOD) v -= MOD;
17        return *this;
18    }
19
20    Mint& operator-=(const Mint& o) {
21        v -= o.v;
22        if (v < 0) v += MOD;
23        return *this;
24    }
25
26    Mint& operator*=(const Mint& o) {
27        v = int(1LL * v * o.v % MOD);
28        return *this;
29    }
30
31    Mint& operator/=(const Mint& o) {
32        v = int(1LL * v * o.inv().v % MOD);
33        return *this;
34    }
35
36    Mint pow(long long p) const {
37        Mint a = *this, res = 1;
38        while (p > 0) {
39            if (p & 1) res *= a;
40            a *= a;
41            p >>= 1;
42        }
43        return res;
44    }
45
46    Mint inv() const { return pow(MOD - 2); }
47
48    friend ostream& operator<<(ostream& os, const Mint& m) {
49        os << m.v;
50        return os;
51    }
52 };
53
54 Mint fact[N], inv[N], invFact[N];
55
56 Mint nCr(int n, int r) {
57     if (r < 0 || r > n)
58         return 0;
59     // return fact[n] / (fact[r] * fact[n - r]);
60     return fact[n] * invFact[r] * invFact[n - r];
61 }
62
63 Mint nPr(int n, int r) {
64     if (r < 0 || r > n)
65         return 0;
66     // return fact[n] / fact[n - r];
67     return fact[n] * invFact[n - r];
68 }
69
70 void build() {
71     fact[0] = invFact[0] = 1;
72     for (int i = 1; i < N; i++) {
73         fact[i] = fact[i - 1] * i;
74         invFact[i] = invFact[i - 1] / i;
75         inv[i] = Mint(i).inv();
76     }
77 }

```

6.2 Multiplicative Functions

6.2.1 EulerTotientPhi

```

1 int phi[N];
2
3 void calculatePhi() {
4     for (int i = 0; i < N; ++i) phi[i] = i & 1 ? i : i / 2;
5     for (int i = 3; i < N; i += 2)
6         if (phi[i] == i)
7             for (int j = i; j < N; j += i) phi[j] -= phi[j] / i;
8 }

```

6.2.2 MobiusFunction

```

1 // Mobius Function (n) Sieve - Time: O(N), Space: O(N)
2 // (n) = 1 if n is square-free with an even number of prime factors
3 // (n) = -1 if n is square-free with an odd number of prime factors
4 // (n) = 0 if n has a squared prime factor
5 struct Mobius {
6     int n;
7     vector<int> mu, primes;
8     vector<bool> is_prime;
9
10    Mobius(int n = 1e7) : n(n), mu(n + 1, 0), is_prime(n + 1, true) {
11        is_prime[0] = is_prime[1] = false;
12        mu[1] = 1;
13
14        for (int i = 2; i <= n; i++) {
15            if (is_prime[i]) {

```

```

16                primes.pb(i);
17                mu[i] = -1;
18            }
19            for (int p : primes) {
20                if ((1ll)i * p > n) break;
21                is_prime[i * p] = false;
22                if (i % p == 0) {
23                    mu[i * p] = 0;
24                    break;
25                } else {
26                    mu[i * p] = -mu[i];
27                }
28            }
29        }
30    }
31 };

```

6.3 Matrices

6.3.1 MatrixExponentiation

```

1 int modSum(ll a, ll b) {
2     if (a < 0)
3         a += MOD;
4     if (b < 0)
5         b += MOD;
6     a += b;
7     if (a >= MOD)
8         a -= MOD;
9     return a;
10 }
11
12 int modProd(ll a, ll b) {
13     if (a < 0)
14         a += MOD;
15     if (b < 0)
16         b += MOD;
17     a *= b;
18     if (a >= MOD)
19         a %= MOD;
20     return a;
21 }
22
23 typedef vector<vector<int>> matrix;
24
25 matrix operator*(const matrix& lhs, const matrix& rhs) {
26     int n = lhs.size();
27     int m = rhs[0].size();
28     int s1 = lhs[0].size(), s2 = rhs.size();
29     assert(s1 == s2);
30     matrix ret(n, vector<int>(m));
31     for (int i = 0; i < n; ++i)
32         for (int j = 0; j < s1; ++j)
33             for (int k = 0; k < m; ++k)
34                 ret[i][k] = modSum(ret[i][k], modProd(lhs[i][j], rhs[j][k]));
35
36     return ret;
37 }
38
39 matrix Identity(int n) {
40     matrix ret(n, vector<int>(n));
41     for (int i = 0; i < n; ++i) {
42         ret[i][i] = 1;
43     }
44     return ret;
45 }
46
47 matrix mat_power(matrix x, ll p) {
48     matrix res = Identity(x.size());
49     while (p) {
50         if (p & 1) res = (res * x);
51         x = (x * x);
52         p >>= 1;
53     }
54     return res;
55 }

```

6.3.2 FastMatrixPower

```

1 const int SZ = 2, mod = 1e9 + 7;
2
3 int modSum(ll a, ll b) {
4     if (a < 0)
5         a += mod;
6     if (b < 0)
7         b += mod;
8     a += b;
9     if (a >= mod)
10        a -= mod;
11    return a;
12 }
13
14 int modProd(ll a, ll b) {
15     if (a < 0)
16         a += mod;
17     if (b < 0)
18         b += mod;
19     a *= b;
20     if (a >= mod)
21         a %= mod;
22     return a;
23 }
24
25 typedef array<array<int, SZ>, SZ> matrix;
26
27 matrix operator*(const matrix &lhs, const matrix &rhs) {
28     matrix ret{};
29     for (int i = 0; i < SZ; ++i)
30         for (int j = 0; j < SZ; ++j)
31             for (int k = 0; k < SZ; ++k)
32                 ret[i][k] = modSum(ret[i][k], modProd(lhs[i][j], rhs[j][k]));
33
34     return ret;
35 }

```

```

35
36 matrix Identity(int n) {
37     matrix ret = {};
38     for (int i = 0; i < n; ++i) {
39         ret[i][i] = 1;
40     }
41     return ret;
42 }
43
44 matrix mat_power(matrix x, ll p) {
45     matrix res = Identity(x.size());
46     while (p) {
47         if (p & 1) res = (res * x);
48         x = (x * x);
49         p >>= 1;
50     }
51     return res;
52 }

```

6.4 Linear Algebra

6.4.1 LinearXorBasis

```

1
2
3 // Linear XOR Basis - Vector Space over GF(2)
4 // Time: O(LOG_BITS) per insert/query
5 struct Basis {
6     vector<ll> basis;
7     int log_bits, sz;
8
9     Basis(int log_bits = 60) : log_bits(log_bits), sz(0), basis(
        log_bits, 0) {}
10
11     bool insert(ll x) {
12         for (int i = log_bits - 1; i >= 0; i--) {
13             if (!(x & (1LL << i))) continue;
14             if (!basis[i]) {
15                 basis[i] = x;
16                 sz++;
17                 return true;
18             }
19             x ^= basis[i];
20         }
21         return false;
22     }
23
24     ll getMax(ll res = 0) const {
25         for (int i = log_bits - 1; i >= 0; i--) {
26             res = max(res, res ^ basis[i]);
27         }
28         return res;
29     }
30 };

```

6.5 Convolution

6.5.1 FFT

```

1 #define rep(aa, bb, cc) for(int aa = bb; aa < cc; aa++)
2 #define sz(a) (int)a.size()
3 typedef complex<double> C;
4 typedef vector<double> vd;
5 void fft(vector<C>& a) {
6     int n = sz(a), L = 31 - __builtin_clz(n);
7     static vector<complex<long double>> R(2, 1);
8     static vector<C> rt(2, 1); // (~ 10% faster if double)
9     for (static int k = 2; k < n; k *= 2) {
10         R.resize(n); rt.resize(n);
11         auto x = polar(1.0L, acos(-1.0L) / k);
12         rep(i, k, 2*k) rt[i] = R[i] = i&1 ? R[i/2] * x : R[i/2];
13     }
14     vi rev(n);
15     rep(i, 0, n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
16     rep(i, 0, n) if (i < rev[i]) swap(a[i], a[rev[i]]);
17     for (int k = 1; k < n; k *= 2)
18         for (int i = 0; i < n; i += 2 * k) rep(j, 0, k) {
19             // C z = rt[j+k] * a[i+j+k]; // (25% faster if hand-
20                 // rolled) /// include-line
21             auto x = (double *)&rt[j+k], y = (double *)&a[i+j+k];
22                 // exclude-line
23             C z(x[0]*y[0] - x[1]*y[1], x[0]*y[1] + x[1]*y[0]);
24                 // exclude-line
25             a[i + j + k] = a[i + j] - z;
26             a[i + j] += z;
27         }
28 }
29
30 template<int M> vi convMod(const vi &a, const vi &b) {
31     if (a.empty() || b.empty()) return {};
32     vi res(sz(a) + sz(b) - 1);
33     int B=32-__builtin_clz(sz(res)), n=1<<B, cut=int(sqrt(M));
34     vector<C> L(n), R(n), outs(n), outl(n);
35     rep(i, 0, sz(a)) L[i] = C((int)a[i] / cut, (int)a[i] % cut);
36     rep(i, 0, sz(b)) R[i] = C((int)b[i] / cut, (int)b[i] % cut);
37     fft(L), fft(R);
38     rep(i, 0, n) {
39         int j = -i & (n - 1);
40         outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
41         outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / ii;
42     }
43     fft(outl), fft(outs);
44     rep(i, 0, sz(res)) {
45         ll av = ll(real(outl[i]).+.5), cv = ll(imag(outs[i]).+.5);
46         ll bv = ll(imag(outl[i]).+.5) + ll(real(outs[i]).+.5);
47         res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
48     }
49     return res;
50 }

```

6.5.2 NTT

```

1 const int MOD = 998244353;

```

```

2 const int ROOT = 3; // Primitive root for 998244353
3
4 long long power(long long base, long long exp) {
5     long long res = 1;
6     base %= MOD;
7     while (exp > 0) {
8         if (exp % 2 == 1) res = (res * base) % MOD;
9         base = (base * base) % MOD;
10        exp /= 2;
11    }
12    return res;
13 }
14
15 long long modInverseFast(long long n) {
16     return power(n, MOD - 2);
17 }
18
19 void ntt(vector<long long>& a, bool invert) {
20     int n = a.size();
21     for (int i = 1, j = 0; i < n; i++) {
22         int bit = n >> 1;
23         for (; j & bit; bit >>= 1) j ^= bit;
24         j ^= bit;
25         if (i < j) swap(a[i], a[j]);
26     }
27     for (int len = 2; len <= n; len <= 1) {
28         long long wlen = power(ROOT, (MOD - 1) / len);
29         if (invert) wlen = modInverseFast(wlen);
30         for (int i = 0; i < n; i += len) {
31             long long w = 1;
32             for (int j = 0; j < len / 2; j++) {
33                 long long u = a[i + j], v = (a[i + j + len / 2] * w) %
                    MOD;
34                 a[i + j] = u + v < MOD ? u + v : u + v - MOD;
35                 a[i + j + len / 2] = u - v >= 0 ? u - v : u - v + MOD;
36                 w = (w * wlen) % MOD;
37             }
38         }
39     }
40     if (invert) {
41         long long n_inv = modInverseFast(n);
42         for (long long& x : a) x = (x * n_inv) % MOD;
43     }
44 }

```

6.5.3 FWHT

```

1 // Fast Walsh-Hadamard Transform (FWHT) - Time: O(N log N)
2 // Computes Bitwise Convolutions: XOR, AND, OR for array of size N = 2^k
3 struct FWHT {
4     enum Type { XOR, AND, OR };
5
6     static void transform(vector<ll>& a, bool invert, Type type = XOR)
7     {
8         int n = a.size();
9         for (int len = 1; 2 * len <= n; len <= 1) {
10             for (int i = 0; i < n; i += 2 * len) {
11                 for (int j = 0; j < len; j++) {
12                     ll u = a[i + j], v = a[i + len + j];
13                     if (type == XOR) {
14                         a[i + j] = u + v;
15                         a[i + len + j] = u - v;
16                     } else if (type == AND) {
17                         a[i + j] = invert ? u - v : u + v;
18                     } else if (type == OR) {
19                         a[i + len + j] = invert ? v - u : u + v;
20                     }
21                 }
22             }
23         }
24         if (type == XOR && invert) {
25             for (ll &x : a) x /= n;
26         }
27     }
28
29     // Computes C[k] = sum_{i op j = k} (A[i] * B[j]) where op in {XOR,
30         AND, OR}
31     static vector<ll> multiply(vector<ll> a, vector<ll> b, Type type =
32         XOR) {
33         int n = 1;
34         while (n < max(a.size(), b.size())) n <= 1;
35         a.resize(n); b.resize(n);
36
37         transform(a, false, type);
38         transform(b, false, type);
39         for (int i = 0; i < n; i++) a[i] *= b[i];
40         transform(a, true, type);
41         return a;
42     }
43 };

```

6.5.4 GeneratingFunctions

```

1 // Formal Power Series (FPS) / Generating Functions - Time: O(N log N)
2 // Operations: Derivative, Integral, Polynomial Inverse, Logarithm ln(F
3     (x))
4 template<int MOD = 998244353>
5 struct PolynomialFPS {
6     typedef vector<ll> Poly;
7
8     static Poly deriv(const Poly& f) {
9         int n = f.size();
10        if (n <= 1) return {0};
11        Poly g(n - 1);
12        for (int i = 1; i < n; i++) g[i - 1] = f[i] * i % MOD;
13        return g;
14    }
15
16    static Poly integ(const Poly& f) {
17        int n = f.size();
18        Poly g(n + 1);
19        for (int i = 0; i < n; i++) g[i + 1] = f[i] * modInverse(i + 1,
20            MOD) % MOD;
21    }
22 }

```

```

19     return g;
20 }
21
22 // Polynomial Inverse mod  $x^{\text{deg}} - O(N \log N)$  via Newton's Method
23 static Poly inv(const Poly& f, int deg) {
24     if (deg == 1) return {modInverse(f[0], MOD)};
25     Poly g = inv(f, (deg + 1) / 2);
26     Poly f_cut(f.begin(), f.begin() + min((int)f.size(), deg));
27
28     //  $g_{\text{new}} = g * (2 - f * g) \bmod x^{\text{deg}}$ 
29     Poly fg = multiply(f_cut, g);
30     fg.resize(deg);
31     for (int i = 0; i < deg; i++) {
32         fg[i] = (i == 0 ? 2 : 0) - fg[i];
33         if (fg[i] < 0) fg[i] += MOD;
34     }
35     Poly res = multiply(g, fg);
36     res.resize(deg);
37     return res;
38 }
39
40 // Polynomial Logarithm  $\ln(F(x)) \bmod x^{\text{deg}} - O(N \log N)$  (requires  $F[0] \neq 0$ )
41 static Poly log(const Poly& f, int deg) {
42     Poly g = integ(multiply(deriv(f), inv(f, deg)));
43     g.resize(deg);
44     return g;
45 }
46
47 private:
48 static Poly multiply(Poly a, Poly b) {
49     // Basic NTT polynomial multiplication helper
50     int n = 1;
51     while (n < a.size() + b.size()) n <= 1;
52     Poly res(a.size() + b.size() - 1, 0);
53     for (int i = 0; i < a.size(); i++) {
54         for (int j = 0; j < b.size(); j++) {
55             res[i + j] = (res[i + j] + a[i] * b[j]) % MOD;
56         }
57     }
58     return res;
59 }
60
61 static ll modInverse(ll a, ll m) {
62     ll x, y;
63     ext_gcd(a, m, x, y);
64     return (x % m + m) % m;
65 }
66
67 static ll ext_gcd(ll a, ll b, ll &x, ll &y) {
68     if (b == 0) { x = 1; y = 0; return a; }
69     ll x1, y1, d = ext_gcd(b, a % b, x1, y1);
70     x = y1; y = x1 - y1 * (a / b);
71     return d;
72 }
73 };

```

7 Strings

7.1 String Matching

7.1.1 KMP

```

1 vector<int> compute_pi(const string &s) {
2     int n = s.size();
3     vector<int> pi(n);
4     for (int i = 1; i < n; i++) {
5         int j = pi[i - 1];
6         while (j > 0 && s[i] != s[j])
7             j = pi[j - 1];
8         if (s[i] == s[j])
9             j++;
10        pi[i] = j;
11    }
12    return pi;
13 }
14
15 vector<int> find_matches(const string &text, const string &pat) {
16     int n = pat.length(), m = text.length();
17     string s = pat + "#" + text;
18     vector<int> pi = compute_pi(s), ans;
19     for (int i = n + 1; i <= n + m; i++) { /* n + 1 is where the text
20         starts */
21         if (pi[i] == n) {
22             ans.push_back(i - 2 * n); /* i - (n - 1) - (n + 1) */
23         }
24     }
25     return ans;
26 }
27
28 void compute_automaton(string s, vector<vector<int>> &aut) {
29     s += '#';
30     int n = s.size();
31     vector<int> pi = compute_pi(s);
32     aut.assign(n, vector<int>(26));
33     for (int i = 0; i < n; i++) {
34         for (int c = 0; c < 26; c++) {
35             if (i > 0 && 'a' + c != s[i])
36                 aut[i][c] = aut[pi[i - 1]][c];
37             else
38                 aut[i][c] = i + ('a' + c == s[i]);
39         }
40     }
41 }

```

7.1.2 RabinKarp

```

1 vector<int> rabin_karp(string const &s, string const &t) {
2     const int p = 31;
3     const int m = 1e9 + 9;
4     int S = s.size(), T = t.size();
5
6     vector<ll> p_pow(max(S, T));

```

```

7     p_pow[0] = 1;
8     for (int i = 1; i < (int) p_pow.size(); i++)
9         p_pow[i] = (p_pow[i - 1] * p) % m;
10
11     vector<ll> h(T + 1, 0);
12     for (int i = 0; i < T; i++)
13         h[i + 1] = (h[i] + (t[i] - 'a' + 1) * p_pow[i]) % m;
14     ll h_s = 0;
15     for (int i = 0; i < S; i++)
16         h_s = (h_s + (s[i] - 'a' + 1) * p_pow[i]) % m;
17
18     vector<int> occurrences;
19     for (int i = 0; i + S - 1 < T; i++) {
20         ll cur_h = (h[i + S] + m - h[i]) % m;
21         if (cur_h == h_s * p_pow[i] % m)
22             occurrences.push_back(i);
23     }
24     return occurrences;
25 }

```

7.1.3 ZFunction

```

1 // z[i] is the length of the longest substring starting from s[i] which
2 // is also a prefix of s
3 vector<int> z_function(string s) {
4     int n = s.length();
5     vector<int> z(n);
6     int l = 0, r = 0;
7     for (int i = 1; i < n; i++) {
8         if (i < r) {
9             z[i] = min(r - i, z[i - l]);
10        }
11        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
12            z[i]++;
13        }
14        if (i + z[i] > r) {
15            l = i;
16            r = i + z[i];
17        }
18    }
19    return z;

```

7.2 Hashing

7.2.1 StringHashing

```

1 int modSum(ll a, ll b) {
2     //complexity: 1
3     if (a < 0)
4         a += MOD;
5     if (b < 0)
6         b += MOD;
7     a += b;
8     if (a >= MOD)
9         a %= MOD;
10    return a;
11 }
12
13 int modProd(ll a, ll b) {
14     //complexity: 1
15     if (a < 0)
16         a += MOD;
17     if (b < 0)
18         b += MOD;
19     a *= b;
20     if (a >= MOD)
21         a %= MOD;
22     return a;
23 }
24
25 int fastPow(int a, ll b) {
26     //complexity: log(b)
27     if (b == 0) return 1;
28     int temp = fastPow(a, b / 2);
29     if (b % 2 == 0)
30         return modProd(temp, temp);
31     return modProd(modProd(a, temp), temp);
32 }
33
34 int modInverse(int a) {
35     //complexity: log(mod)
36     return fastPow(a, MOD - 2);
37 }
38
39
40
41 struct Hash {
42     vector<pair<int, int>> hash_value = {{0, 0}};
43     const pair<int, int> p = {67, 97};
44     vector<pair<int, int>> p_pow = {{1, 1}};
45     vector<pair<int, int>> inv = {{1, 1}};
46
47     void compute(string const &s) {
48         for (char c: s) {
49             if (c >= 'a' && c <= 'z') {
50                 c = c - 'a' + 1;
51             } else if (c >= 'A' && c <= 'Z') {
52                 c = c - 'A' + 30;
53             } else {
54                 c = c - '0' + 60;
55             }
56             int x = modSum(hash_value.back().st, modProd(c, p_pow.back().st));
57             int a = modProd(p_pow.back().st, p.st);
58
59             int y = modSum(hash_value.back().nd, modProd(c, p_pow.back().nd));
60             int b = modProd(p_pow.back().nd, p.nd);
61
62             hash_value.emplace_back(x, y);
63             p_pow.emplace_back(a, b);
64             if (inv.size() == 1) {
65                 inv.emplace_back(modInverse(a), modInverse(b));

```



```

66         } else {
67             inv.emplace_back(modProd(inv[l].st, inv.back().st),
                                   modProd(inv[l].nd, inv.back().nd));
68         }
69     }
70 }
71
72 pair<int, int> get(int l, int r) {
73     l++, r++;
74     pair<int, int> ans;
75     ans.st = modSum(hash_value[r].st, -hash_value[l - 1].st);
76     ans.st = modProd(ans.st, inv[l].st);
77
78     ans.nd = modSum(hash_value[r].nd, -hash_value[l - 1].nd);
79     ans.nd = modProd(ans.nd, inv[l].nd);
80
81     return ans;
82 }
83
84 };

```

7.3 Aho Corasick

7.3.1 AhoCorasick

```

1 struct Mapper {
2     int operator()(char c) const { return c - 'a'; }
3 };
4
5 template <int ALPHABET, typename Mapper>
6 struct Aho {
7     struct Node {
8         array<int, ALPHABET> nxt;
9         int link = 0;
10        int terminal_idx = -1; // -1 if not a terminal, else idx of
11                               string insertion
12        int exit_link = 0; // Points to the nearest terminal suffix
13        int depth = 0; // Size of the prefix the node represents
14        Node() { nxt.fill(0); }
15    };
16
17    vector<Node> t;
18    vector<int> bfs_order;
19    Mapper get_idx; // Instance of the mapping function
20    int word_count = 0; // Tracks the order of inserted strings
21
22    // Constructor optionally accepts a custom mapper instance
23    Aho(Mapper mapper = Mapper()) : get_idx(mapper) {
24        t.emplace_back();
25    }
26
27    int insert(const string& p) {
28        int u = 0;
29        for (char c : p) {
30            int v = get_idx(c);
31            if (!t[u].nxt[v]) {
32                t[u].nxt[v] = t.size();
33                t.emplace_back();
34                t.back().depth = t[u].depth + 1; // New node's depth is
35                                                  parent's depth + 1
36            }
37            u = t[u].nxt[v];
38        }
39
40        // If this exact string hasn't been inserted before, assign it
41        // the current index
42        if (t[u].terminal_idx == -1) {
43            t[u].terminal_idx = word_count;
44        }
45        word_count++; // Increment for the next insertion
46
47        return u;
48    }
49
50    void build() {
51        queue<int> q;
52        for (int c = 0; c < ALPHABET; ++c) {
53            if (t[0].nxt[c]) {
54                q.push(t[0].nxt[c]);
55            }
56        }
57
58        while (!q.empty()) {
59            int u = q.front();
60            q.pop();
61            bfs_order.push_back(u);
62
63            for (int c = 0; c < ALPHABET; ++c) {
64                int v = t[u].nxt[c];
65                if (v) {
66                    t[v].link = t[t[u].link].nxt[c];
67
68                    // Compute the exit link:
69                    // If the immediate suffix link is a terminal,
70                    // point to it.
71                    // Otherwise, copy its exit link.
72                    t[v].exit_link = (t[t[v].link].terminal_idx != -1)
73                        ? t[v].link : t[t[v].link].exit_link;
74
75                    q.push(v);
76                }
77            }
78            else {
79                t[u].nxt[c] = t[t[u].link].nxt[c];
80            }
81        }
82    }
83
84    int advance(int u, char c) {
85        return t[u].nxt[get_idx(c)];
86    }
87 };

```

7.4 Suffix Structures

7.4.1 SuffixArray

```

1 struct SuffixArray {
2     string S;
3     // sa is the suffix array with the empty suffix being sa[0]
4     // lcp[i] holds the lcp between sa[i], sa[i - 1]
5     vector<int> logs, sa, lcp, rank;
6     vector<vector<int>> table;
7
8     SuffixArray() {};
9
10    SuffixArray(string &s, int lim = 256) {
11        S = s;
12        int n = s.size() + 1, k = 0, a, b;
13        vector<int> c(s.begin(), s.end() + 1), tmp(n), frq(max(n, lim))
14        ;
15        c.back() = 0; // 0 is less than any character
16        sa = lcp = rank = tmp, iota(sa.begin(), sa.end(), 0);
17        for (int j = 0, p = 0; p < n; j = max(1, j * 2), lim = p) {
18            p = j, iota(tmp.begin(), tmp.end(), n - j);
19            for (int i = 0; i < n; i++) {
20                if (sa[i] >= j)
21                    tmp[p++] = sa[i] - j;
22            }
23
24            fill(frq.begin(), frq.end(), 0);
25
26            for (int i = 0; i < n; i++) frq[c[i]]++;
27            for (int i = 1; i < lim; i++) frq[i] += frq[i - 1];
28            for (int i = n; i--;) sa[--frq[c[tmp[i]]]] = tmp[i];
29
30            swap(c, tmp), p = 1, c[sa[0]] = 0;
31            for (int i = 1; i < n; i++)
32                a = sa[i - 1], b = sa[i], c[b] = (tmp[a] == tmp[b] &&
33                    tmp[a + j] == tmp[b + j]) ? p - 1 : p++;
34        }
35
36        for (int i = 1; i < n; i++) rank[sa[i]] = i;
37        for (int i = 0, j; i < n - 1; lcp[rank[i++]] = k)
38            for (k &&k--, j = sa[rank[i] - 1];
39                 s[i + k] == s[j + k];
40                 k++);
41    }
42
43    void preLcp() {
44        int n = S.size() + 1;
45        logs = vector<int>(n + 5);
46        for (int i = 2; i < n + 5; ++i) {
47            logs[i] = logs[i / 2] + 1;
48        }
49        table = vector<vector<int>>(n, vector<int>(20));
50        for (int i = 0; i < n; ++i) {
51            table[i][0] = lcp[i];
52        }
53
54        for (int j = 1; j <= logs[n]; ++j) {
55            for (int i = 0; i <= n - (1 << j); ++i) {
56                table[i][j] = min(table[i][j - 1], table[i + (1 << (j -
57                    1))][j - 1]);
58            }
59        }
60    }
61
62    int queryLcp(int i, int j) {
63        // if (i == j) return (int) S.size() - i;
64        // i = rank[i], j = rank[j];
65        if (i == j) return (int) S.size() - sa[i];
66        if (i > j)
67            swap(i, j);
68        ++i;
69        int len = logs[j - i + 1];
70        return min(table[i][len], table[j - (1 << len) + 1][len]);
71    }
72 };

```

7.4.2 SuffixAutomaton

```

1 struct SuffixAutomaton {
2     struct Node {
3         int len = 0, link = -1;
4         map<char, int> next;
5     };
6
7     vector<Node> st;
8     int sz = 1, last = 0;
9
10    SuffixAutomaton(const string& s) {
11        st.resize(s.length() * 2);
12        st[0].len = 0;
13        st[0].link = -1;
14        for (char c : s) extend(c);
15    }
16
17    void extend(char c) {
18        int cur = sz++;
19        st[cur].len = st[last].len + 1;
20        int p = last;
21        while (p != -1 && !st[p].next.count(c)) {
22            st[p].next[c] = cur;
23            p = st[p].link;
24        }
25        if (p == -1) {
26            st[cur].link = 0;
27        }
28        else {
29            int q = st[p].next[c];
30            if (st[p].len + 1 == st[q].len) {
31                st[cur].link = q;
32            }
33            else {
34                int clone = sz++;
35                st[clone].len = st[p].len + 1;
36                st[clone].next = st[q].next;
37                st[clone].link = st[q].link;

```

```

36         while (p != -1 && st[p].next[c] == q) {
37             st[p].next[c] = clone;
38             p = st[p].link;
39         }
40         st[q].link = st[cur].link = clone;
41     }
42     }
43     last = cur;
44 }
45 };

```

7.5 Palindromes

7.5.1 Manacher

```

1 struct Manacher {
2     vector<int> p;
3
4     // Returns a vector where p[i] is the radius of the longest
5     // palindrome around center i
6     // Centers are interleaved: 0 is before string, 1 is first char, 2
7     // is between 1st & 2nd, etc.
8     Manacher(string s) {
9         string t = "#";
10        for (char c : s) {
11            t += c;
12            t += "#";
13        }
14        int n = t.size();
15        p.assign(n, 0);
16        int c = 0, r = 0;
17        for (int i = 0; i < n; i++) {
18            int mirror = 2 * c - i;
19            if (i < r) p[i] = min(r - i, p[mirror]);
20            while (i - 1 - p[i] >= 0 && i + 1 + p[i] < n && t[i - 1 - p[i]] == t[i + 1 + p[i]]) {
21                p[i]++;
22            }
23            if (i + p[i] > r) {
24                c = i;
25                r = i + p[i];
26            }
27        }
28
29        // Check if substring [l, r] (0-based) is a palindrome in O(1)
30        bool is_palindrome(int l, int r) {
31            int len = r - l + 1;
32            int center = l + r + 1;
33            return p[center] >= len;
34        }
35    };

```

8 Dynamic Programming (DP)

8.1 DP Optimizations

8.1.1 DivideAndConquerDP

```

1 // Divide & Conquer DP Optimization - Time: O(K * N log N), Space: O(N)
2 // Applies when DP transition is dp[k][i] = min_{j < i} {dp[k-1][j] +
3 // cost(j+1, i)}
4 // Condition: opt(i, j) <= opt(i, j + 1) (Monotonicity of optimal split
5 // points)
6 int a[N];
7 ll curCost;
8 int curL = 0, curR = -1, freq[N];
9 pair<int, ll> pre[N], cur[N];
10 // Modify at will :
11 int lo(int ind) { return 1; }
12 int hi(int ind) { return ind; }
13
14 void add(int val) {
15     curCost += freq[val]++;
16 }
17
18 void remove(int val) {
19     curCost -= --freq[val];
20 }
21
22 ll Cost(int l, int r) {
23     while (curR < r)
24         add(a[++curR]);
25     while (curL > l)
26         add(a[--curL]);
27     while (curR > r)
28         remove(a[curR--]);
29     while (curL < l)
30         remove(a[curL++]);
31     return curCost;
32 }
33
34 ll f(int ind, int k) { return pre[k - 1].nd + Cost(k, ind); }
35 void store(int ind, int k, ll v) { cur[ind] = pair(k, v); }
36
37 void rec(int L, int R, int LO, int HI) {
38     if (L > R) return;
39     int mid = (L + R) >> 1;
40     pair best(llinf, LO);
41     for (int k = max(LO, lo(mid)); k <= min(HI, hi(mid)); ++k)
42         best = min(best, pair(f(mid, k), k));
43     store(mid, best.second, best.first);
44     rec(L, mid - 1, LO, best.second);
45     rec(mid + 1, R, best.second, HI);
46 }
47
48 void solve(int L, int R) { rec(L, R, -inf, inf); }

```

8.1.2 CHT

```

1 //To query the maximum value of the function for all x in [l, r], query
2 //only two points: l, r because the functions are convex.
3 struct Line {

```

```

4     ll m, b;
5     mutable function<const Line *(> succ;
6
7     bool operator<(const Line &other) const {
8         return m < other.m;
9     }
10
11     bool operator<(const ll &x) const {
12         const Line *s = succ();
13         if (!s)
14             return 0;
15         return b - s->b < (s->m - m) * x;
16     }
17 };
18
19 // will maintain upper hull for maximum
20 struct HullDynamic : public multiset<Line, less<>> {
21     bool bad(iterator y) {
22         auto z = next(y);
23         if (y == begin()) {
24             if (z == end())
25                 return 0;
26             return y->m == z->m && y->b <= z->b;
27         }
28         auto x = prev(y);
29         if (z == end())
30             return y->m == x->m && y->b <= x->b;
31         return (ld)(x->b - y->b) * (z->m - y->m) >= (ld)(y->b - z->b)
32             * (y->m - x->m);
33     }
34
35     void insert_line(ll m, ll b) {
36         // for minimum
37         m *= -1;
38         b *= -1;
39         auto y = insert({m, b});
40         y->succ = [=] { return next(y) == end() ? 0 : &*next(y); };
41         if (bad(y)) {
42             erase(y);
43             return;
44         }
45         while (next(y) != end() && bad(next(y)))
46             erase(next(y));
47         while (y != begin() && bad(prev(y)))
48             erase(prev(y));
49     }
50
51     ll query(ll x) {
52         auto l = *lower_bound(x);
53         // for minimum
54         return -(l.m * x + l.b);
55     }
56 };

```

8.1.3 CHTRollback

```

1 // Rollback Convex Hull Trick - Space: O(N), Add: O(log N), Query: O(
2 // log N), Rollback: O(1)
3 // Supports inserting monotonic lines with history stack rollback for
4 // tree/dsu DP
5 // Template parameters:
6 // IS_MAX: Set to true for Maximum queries, false for Minimum queries.
7 // IS_INC: Set to true if slopes added are strictly Increasing, false
8 // for Decreasing.
9 template<bool IS_MAX = false, bool IS_INC = false>
10 struct RollbackCHT {
11     struct Line {
12         ll m, c;
13         Line() : m(0), c(llinf) {}
14         Line(ll m, ll c) : m(m), c(c) {}
15         ll eval(ll x) const { return m * x + c; }
16     };
17
18     struct History {
19         int pos;
20         int sz;
21         Line old_line;
22     };
23
24     vector<Line> st;
25     vector<History> hist;
26     int sz;
27
28     RollbackCHT(int max_nodes) {
29         st.resize(max_nodes);
30         sz = 0;
31     }
32
33     bool redundant(Line l1, Line l2, Line l3) {
34         __int128 dx1 = l1.m - l2.m;
35         __int128 dc1 = l2.c - l1.c;
36         __int128 dx2 = l2.m - l3.m;
37         __int128 dc2 = l3.c - l2.c;
38         return dc1 * dx2 >= dc2 * dx1;
39     }
40
41     void add(ll m, ll c) {
42         // The magic happens here: automatically formats internal state
43         ll m_int = IS_INC ? -m : m;
44         ll c_int = IS_MAX ? -c : c;
45
46         Line l_new(m_int, c_int);
47         int pos = sz;
48         int low = 1, high = sz - 1;
49
50         while (low <= high) {
51             int mid = low + (high - low) / 2;
52             if (redundant(st[mid - 1], st[mid], l_new)) {
53                 pos = mid;
54                 high = mid - 1;
55             } else {
56                 low = mid + 1;
57             }
58         }

```

```

55     }
56
57     hist.push_back({pos, sz, st[pos]});
58     st[pos] = l_new;
59     sz = pos + 1;
60 }
61
62 void rollback() {
63     if (hist.empty()) return;
64     History h = hist.back();
65     hist.pop_back();
66     sz = h.sz;
67     st[h.pos] = h.old_line;
68 }
69
70 ll query(ll x) {
71     if (sz == 0) return IS_MAX ? -llinf : llinf;
72
73     // The magic happens here: matches x to the internal geometry
74     ll x_int = (IS_INC != IS_MAX) ? -x : x;
75
76     int low = 0, high = sz - 2;
77     int best = sz - 1;
78
79     while (low <= high) {
80         int mid = low + (high - low) / 2;
81         if (st[mid].eval(x_int) <= st[mid + 1].eval(x_int)) {
82             best = mid;
83             high = mid - 1;
84         } else {
85             low = mid + 1;
86         }
87     }
88
89     ll ans = st[best].eval(x_int);
90     return IS_MAX ? -ans : ans;
91 }
92 };

```

8.1.4 LiChaoTree

```

1 // Li Chao Tree - Space:  $O(N \log X)$ , Insert Line:  $O(\log X)$ , Query:  $O(\log X)$ 
2 // Dynamic segment tree evaluating line functions  $y = m * x + b$  for
3 // arbitrary queries x
4 // To get an instance : node *root = EMPTY;
5 typedef pair<long long, long long> line;
6 int start_x, end_x;
7 line neutral{0, -1e18};
8
9 ll sub(const line &l, ll x) {
10     return l.first * x + l.second;
11 }
12
13 double inter(const line &l1, const line &l2) {
14     return (l2.second - l1.second) / (l1.first - l2.first - .0);
15 }
16
17 extern struct node *const EMPTY;
18 struct node {
19     line l;
20     node *lf, *rt;
21     node() : l(neutral), lf(this), rt(this) {}
22     node(line l) : l(l), lf(EMPTY), rt(EMPTY) {}
23 };
24 node *const EMPTY = new node;
25
26 void insert(line l, int lx, int rx, node *&cur, ll ns = start_x, ll ne = end_x) {
27     if (rx < ns || lx > ne) return;
28     if (ns >= lx && ne <= rx) {
29         if (cur == EMPTY) {
30             cur = new node(l);
31             return;
32         }
33         if (l.first == cur->l.first) {
34             cur->l = max(cur->l, l);
35             return;
36         }
37         auto x = inter(l, cur->l);
38         if (x < ns || x > ne) {
39             if (sub(l, ns) > sub(cur->l, ns)) cur->l = l;
40             return;
41         }
42         ll mid = ns + (ne - ns) / 2;
43         if (x < mid) {
44             if (sub(l, ne) > sub(cur->l, ne)) swap(l, cur->l);
45             insert(l, lx, rx, cur->lf, ns, mid);
46         } else {
47             if (sub(l, ns) > sub(cur->l, ns)) swap(l, cur->l);
48             insert(l, lx, rx, cur->rt, mid + 1, ne);
49         }
50         return;
51     }
52
53     if (cur == EMPTY) {
54         cur = new node(neutral);
55     }
56     ll mid = ns + (ne - ns) / 2;
57     insert(l, lx, rx, cur->lf, ns, mid);
58     insert(l, lx, rx, cur->rt, mid + 1, ne);
59 }
60
61 ll query(ll x, node *cur, ll ns = start_x, ll ne = end_x) {
62     auto ret = sub(cur->l, x);
63     if (x < ns || x > ne)
64         return sub(neutral, x);
65     if (ns == ne)
66         return ret;
67     ll mid = ns + (ne - ns) / 2;
68     if (x <= mid)
69         return max(ret, query(x, cur->lf, ns, mid));
70     return max(ret, query(x, cur->rt, mid + 1, ne));

```

```

71 }

```

8.1.5 PersistentLiChaoTree

```

1 // Persistent Li Chao Tree - Space:  $O(N \log X)$ , Insert Line:  $O(\log X)$ ,
2 // Query:  $O(\log X)$ 
3 // Version-persistent Li Chao Segment Tree for tree / history DP
4 // queries
5 const ll maxN = 1e7 + 5;
6
7 struct Line {
8     ll m, c;
9
10     Line() : m(0), c(llinf) {}
11     Line(ll m, ll c) : m(m), c(c) {}
12 };
13
14 ll sub(ll x, Line l) {
15     return x * l.m + l.c;
16 }
17
18 // Persistent Li Chao
19 struct Node {
20     // range I am responsible for
21     Line line;
22     Node *left, *right;
23
24     Node() {
25         left = right = NULL;
26     }
27
28     Node(ll m, ll c) {
29         line = Line(m, c);
30         left = right = NULL;
31     }
32
33 void extend(int l, int r) {
34     if (left == NULL && l != r) {
35         left = new Node();
36         right = new Node();
37     }
38
39     Node *copy(Node *node) {
40         Node *newNode = new Node;
41         newNode->left = node->left;
42         newNode->right = node->right;
43         newNode->line = node->line;
44         return newNode;
45     }
46
47     Node *add(Line toAdd, int l, int r) {
48         int mid = (l + r) / 2;
49         Node *cur = copy(this);
50         if (l == r) {
51             if (sub(l, toAdd) < sub(l, cur->line))
52                 swap(toAdd, cur->line);
53             return cur;
54         }
55         bool lef = sub(l, toAdd) < sub(l, cur->line);
56         bool midE = sub(mid + 1, toAdd) < sub(mid + 1, cur->line);
57         if (midE)
58             swap(cur->line, toAdd);
59         cur->extend(l, r);
60         if (lef != midE) {
61             cur->left = cur->left->add(toAdd, l, mid);
62         } else {
63             if (mid + 1 > r)
64                 return cur;
65             cur->right = cur->right->add(toAdd, mid + 1, r);
66         }
67         return cur;
68     }
69
70     Node *add(Line toAdd) {
71         return add(toAdd, -maxN + 1, maxN - 1);
72     }
73
74 ll query(ll x, int l, int r) {
75     int mid = (l + r) / 2;
76     if (l == r || left == NULL)
77         return sub(x, line);
78     extend(l, r);
79     if (x <= mid)
80         return min(sub(x, line), left->query(x, l, mid));
81     else
82         return min(sub(x, line), right->query(x, mid + 1, r));
83 }
84
85 ll query(ll x) {
86     return query(x, -maxN + 1, maxN - 1);
87 }
88
89 void clear() {
90     if (left != NULL) {
91         left->clear();
92         right->clear();
93     }
94     delete this;
95 }
96 };
97
98 Node *tree[N];

```

8.1.6 KnuthDP

```

1 // Knuth's DP Optimization - Time:  $O(N^2)$ , Space:  $O(N^2)$ 
2 // Optimizes interval DP:  $dp[i][j] = \min_{i \leq k < j} (dp[i][k] + dp[k+1][j]) + cost(i, j)$ 
3 // Applicable when  $opt[i][j-1] \leq opt[i][j] \leq opt[i+1][j]$ 
4 struct KnuthDP {
5     int n;
6     vector<vector<ll>>> dp;

```

```

7     vector<vector<int>>> opt;
8
9     KnuthDP(int n = 0) : n(n), dp(n + 2, vector<ll>(n + 2, 0)), opt(n +
10         2, vector<int>(n + 2, 0)) {}
11
12     // User-defined cost function cost(i, j)
13     ll solve(function<ll(int, int)> cost) {
14         for (int i = 1; i <= n; i++) {
15             opt[i][i] = i;
16
17             for (int len = 2; len <= n; len++) {
18                 for (int i = 1; i + len - 1 <= n; i++) {
19                     int j = i + len - 1;
20                     dp[i][j] = llinf;
21                     int L = opt[i][j - 1], R = opt[i + 1][j];
22                     for (int k = L; k <= min(R, j - 1); k++) {
23                         ll cur = dp[i][k] + dp[k + 1][j] + cost(i, j);
24                         if (cur < dp[i][j]) {
25                             dp[i][j] = cur;
26                             opt[i][j] = k;
27                         }
28                     }
29                 }
30             }
31         }
32         return dp[1][n];
33     };

```

8.1.7 AliensTrick

```

1 // Alien's Trick (WNS / Penalty Binary Search) - Time: O(N log(
2 // PENALTY_RANGE))
3 // Solves exact K-element selection / partition DP when cost function f
4 // (k) is strictly convex
5 // Adds a penalty lambda per selected item to drop the k constraint
6 struct AliensTrick {
7     // Binary searches optimal penalty lambda to choose exactly K items
8     // Returns minimum cost to choose exactly K items
9     static ll solve(int n, int k, ll min_penalty, ll max_penalty,
10         function<pair<ll, int>(ll)> dp_solver) {
11         ll l = min_penalty, r = max_penalty, best_lambda = 0;
12
13         while (l <= r) {
14             ll mid = l + (r - l) / 2;
15             auto [cost, count] = dp_solver(mid);
16             if (count >= k) {
17                 best_lambda = mid;
18                 l = mid + 1; // Increase penalty if we used too many
19                             // items
20             } else {
21                 r = mid - 1; // Decrease penalty if we used too few
22                             // items
23             }
24         }
25
26         // True cost = cost_with_penalty - (k * best_lambda)
27         auto [cost, count] = dp_solver(best_lambda);
28         return cost - (ll)k * best_lambda;
29     }
30 };

```

8.2 Bitmask DP

8.2.1 SOS_DP

```

1 // Computes the sum of all subsets for every bitmask in O(N * 2^N)
2 vector<long long> SOS_DP(int n_bits, const vector<long long>& a) {
3     int max_mask = 1 << n_bits;
4     vector<long long> dp = a; // dp[mask] initially holds exactly the
5     // value for mask
6
7     for (int i = 0; i < n_bits; ++i) {
8         for (int mask = 0; mask < max_mask; ++mask) {
9             if (mask & (1 << i)) {
10                 dp[mask] += dp[mask ^ (1 << i)];
11             }
12         }
13     }
14     return dp; // dp[mask] now contains sum of all submasks of mask

```

9 Trees

9.1 Lowest Common Ancestor (LCA)

9.1.1 LCA

```

1 struct LCA {
2     static const int B = 21;
3     vector<array<int, 2>> tour;
4     vector<array<array<int, 2>, B>> T;
5     vector<int> d, in, lg;
6
7     LCA(int n, auto &adj): d(n + 1), in(n + 1), lg(n * 2 + 1) {
8         tour.reserve(n * 2);
9         T.resize(n);
10        dfs(0, 0, adj);
11        for (int i = 0; i < n; ++i) lg[i] = __lg(i), T[i][0] = tour[i];
12        for (int j = 1; j < B; ++j)
13            for (int i = 0; i + (1 << j) - 1 < n; ++i)
14                T[i][j] = min(T[i][j - 1], T[i + (1 << j - 1)][j - 1]);
15    }
16
17    void dfs(int u, int p, auto &adj) {
18        in[u] = size(tour);
19        tour.push_back({d[u], u});
20        for (int v: adj[u]) {
21            if (v == p) continue;
22            d[v] = d[u] + 1;
23            dfs(v, u, adj);
24            tour.push_back({d[u], u});
25        }

```

```

26    }
27
28    int get(int u, int v) {
29        int l = min(in[u], in[v]), r = max(in[u], in[v]);
30        int j = lg[r - l + 1];
31        return min(T[l][j], T[r - (1 << j) + 1][j])[1];
32    }
33
34    int dist(int u, int v) { return d[u] + d[v] - 2 * d[get(u, v)]; }
35 };

```

9.2 DSU on Tree

9.2.1 DSUOnTree

```

1 class DSUOnTree {
2 private:
3     struct Query {
4         int idx, x;
5     };
6
7     int n;
8     vector<vector<int>>> adj;
9     vector<int> sz, depth;
10    vector<bool> big;
11    vector<vector<Query>>> queries;
12    vector<int> ans;
13
14    // =====
15    // PROBLEM SPECIFIC STATE VARIABLES
16    // =====
17    vector<char> c; // Example: Array of characters per node
18
19    // Example state variables
20    // int distinct_count = 0;
21    // vector<int> freq;
22
23    void add(int u, int p) {
24        // --- ADD LOGIC HERE ---
25        // Example: freq[c[u] - 'a']++;
26
27        for (int v : adj[u]) {
28            if (v == p || big[v]) continue; // Skip parent and heavy
29                                           // child[cite: 4]
30            add(v, u);
31        }
32    }
33
34    void remove(int u, int p) {
35        // --- REMOVE LOGIC HERE ---
36        // Example: freq[c[u] - 'a']--;
37
38        for (int v : adj[u]) {
39            if (v == p || big[v]) continue; // Skip parent and heavy
40                                           // child[cite: 4]
41            remove(v, u);
42        }
43    }
44
45    int get_answer(int u, int x = 0) {
46        // --- QUERY ANSWER LOGIC HERE ---
47        return 0;
48    }
49
50    void pre(int u, int p = 0) {
51        sz[u] = 1;
52        depth[u] = depth[p] + 1;
53        for (int v : adj[u]) {
54            if (v == p) continue;
55            pre(v, u);
56            sz[u] += sz[v]; // Compute subtree size[cite: 4]
57        }
58    }
59
60    void dfs(int u, int p = 0, bool keep = false) {
61        int mx = -1, bigChild = -1;
62
63        // Find the heavy child[cite: 4]
64        for (int v : adj[u]) {
65            if (v == p) continue;
66            if (sz[v] > mx) {
67                mx = sz[v];
68                bigChild = v;
69            }
70        }
71
72        // Process light children and clear them[cite: 4]
73        for (int v : adj[u]) {
74            if (v == p || v == bigChild) continue;
75            dfs(v, u, false);
76        }
77
78        // Process heavy child and keep it[cite: 4]
79        if (bigChild != -1) {
80            dfs(bigChild, u, true);
81            big[bigChild] = true;
82        }
83
84        // Add current node and its light children[cite: 4]
85        add(u, p);
86
87        // Answer all queries for the current node[cite: 4]
88        for (const auto& q : queries[u]) {
89            ans[q.idx] = get_answer(u, q.x);
90        }
91
92        // Reset the big child flag[cite: 4]
93        if (bigChild != -1) {
94            big[bigChild] = false;
95        }
96
97        // Clear state if this node is not meant to be kept[cite: 4]
98        if (!keep) {

```

```

98         remove(u, p);
99     }
100 }
101
102 public:
103     // Initialize with 1-based indexing in mind
104     DSUOnTree(int nodes, const vector<char>& chars) : n(nodes), c(chars)
105     {
106         adj.assign(n + 1, vector<int>());
107         sz.assign(n + 1, 0);
108         depth.assign(n + 1, 0);
109         big.assign(n + 1, false);
110         queries.assign(n + 1, vector<Query>());
111         // Initialize problem-specific arrays here (e.g., freq.assign(26, 0));
112     }
113
114     void add_edge(int u, int v) {
115         adj[u].push_back(v);
116         adj[v].push_back(u);
117     }
118
119     void add_query(int node, int query_idx, int x = 0) {
120         queries[node].push_back({query_idx, x});
121     }
122
123     vector<int> solve(int root, int num_queries) {
124         ans.assign(num_queries, 0);
125         pre(root, 0); // Precalculate sizes and depths[cite: 4]
126         dfs(root, 0, false); // Run the sack algorithm[cite: 4]
127         return ans;
128     };

```

9.3 Heavy Light Decomposition (HLD)

9.3.1 HLD

```

1 class HLD {
2 public:
3     vector<int> par, sz, head, tin, tout, who, depth;
4
5     int dfs1(int u, vector<vector<int>> &adj) {
6         for (int &v: adj[u]) {
7             if (v == par[u]) continue;
8             depth[v] = depth[u] + 1;
9             par[v] = u;
10            sz[u] += dfs1(v, adj);
11            if (sz[v] > sz[adj[u][0]] || adj[u][0] == par[u]) swap(v, adj[u][0]);
12        }
13        return sz[u];
14    }
15
16    void dfs2(int u, int &timer, const vector<vector<int>> &adj) {
17        tin[u] = timer++;
18        for (int v: adj[u]) {
19            if (v == par[u]) continue;
20            head[v] = (timer == tin[u] + 1 ? head[u] : v);
21            dfs2(v, timer, adj);
22        }
23        tout[u] = timer - 1;
24    }
25
26    HLD(vector<vector<int>> &adj, int r = 0)
27        : par(adj.size(), -1), sz(adj.size(), 1), head(adj.size(), r),
28          tin(adj.size()), tout(adj.size()),
29          who(adj.size()),
30          depth(adj.size()) {
31        dfs1(r, adj);
32        int x = 0;
33        dfs2(r, x, adj);
34        for (int i = 0; i < adj.size(); ++i) who[tin[i]] = i;
35    }
36
37    vector<pair<int, int>> path(int u, int v) {
38        vector<pair<int, int>> res;
39        for (; v = par[head[v]]) {
40            if (depth[head[u]] > depth[head[v]]) swap(u, v);
41            if (head[u] != head[v]) {
42                res.emplace_back(tin[head[v]], tin[v]);
43            } else {
44                if (depth[u] > depth[v]) swap(u, v);
45                res.emplace_back(tin[u], tin[v]);
46                return res;
47            }
48        }
49
50        pair<int, int> subtree(int u) {
51            return {tin[u], tout[u]};
52        }
53
54        int dist(int u, int v) {
55            return depth[u] + depth[v] - 2 * depth[lca(u, v)];
56        }
57
58        int lca(int u, int v) {
59            for (; v = par[head[v]]) {
60                if (depth[head[u]] > depth[head[v]]) swap(u, v);
61                if (head[u] == head[v]) {
62                    if (depth[u] > depth[v]) swap(u, v);
63                    return u;
64                }
65            }
66        }
67
68        bool isAncestor(int u, int v) {
69            return tin[u] <= tin[v] && tout[u] >= tout[v];
70        }
71    };

```

9.4 Centroid Decomposition

9.4.1 Centroid

```

1 class CentroidDecomposition {
2 private:
3     int n;
4     vector<vector<int>> adj;
5     vector<int> sz, par, nodes;
6     vector<bool> vis;
7
8     int dfsSz(int u, int p) {
9         sz[u] = 1;
10        for (int v : adj[u]) {
11            if (vis[v] || v == p) continue;
12            sz[u] += dfsSz(v, u);
13        }
14        return sz[u];
15    }
16
17    int getCentroid(int u, int p, int total_nodes) {
18        for (int v : adj[u]) {
19            if (vis[v] || v == p) continue;
20            if (sz[v] * 2 > total_nodes) {
21                return getCentroid(v, u, total_nodes);
22            }
23        }
24        return u;
25    }
26
27    void collect(int u, int p) {
28        nodes.push_back(u);
29        for (int v : adj[u]) {
30            if (vis[v] || v == p) continue;
31            collect(v, u);
32        }
33    }
34
35    void build(int u, int p) {
36        int total_nodes = dfsSz(u, p);
37        int centroid = getCentroid(u, p, total_nodes);
38
39        if (p != -1) {
40            par[centroid] = p;
41        } else {
42            par[centroid] = centroid;
43        }
44
45        vis[centroid] = true;
46
47        // =====
48        // PROBLEM SPECIFIC PROCESSING
49        // =====
50        // Process paths going through the 'centroid' here
51        for (int v : adj[centroid]) {
52            if (vis[v]) continue;
53            nodes.clear();
54            collect(v, centroid);
55
56            // e.g., Update answers using the nodes collected from this subtree branch
57        }
58        // =====
59
60        // Recursively decompose the remaining components
61        for (int v : adj[centroid]) {
62            if (vis[v]) continue;
63            build(v, centroid);
64        }
65    }
66
67 public:
68     // Initialize with 1-based indexing
69     CentroidDecomposition(int nodes_count) : n(nodes_count) {
70         adj.assign(n + 1, vector<int>());
71         sz.assign(n + 1, 0);
72         par.assign(n + 1, 0);
73         vis.assign(n + 1, false);
74     }
75
76     void add_edge(int u, int v) {
77         adj[u].push_back(v);
78         adj[v].push_back(u);
79     }
80
81     // Start decomposition, usually from node 1
82     void decompose(int root = 1) {
83         build(root, -1);
84     }
85
86     // Returns the parent of a node in the centroid tree
87     int get_centroid_parent(int u) const {
88         return par[u];
89     }
90 };

```

9.5 Tree Diameter

9.5.1 TreeDiameter

```

1 // Tree Diameter via 2-BFS - Time: O(V + E), Space: O(V + E)
2 // Computes diameter length, endpoints (u, v), and the full diameter path
3 struct TreeDiameter {
4     int n;
5     vector<vector<int>> adj;
6     vector<int> dist, par;
7
8     TreeDiameter(int n = 0) : n(n), adj(n + 1), dist(n + 1), par(n + 1) {}
9
10    void add_edge(int u, int v) {
11        adj[u].pb(v);
12        adj[v].pb(u);

```

```

13     }
14
15     int bfs(int src) {
16         fill(all(dist), -1);
17         queue<int> q;
18         q.push(src);
19         dist[src] = 0;
20         par[src] = -1;
21
22         int farthest = src;
23         while (!q.empty()) {
24             int u = q.front();
25             q.pop();
26             if (dist[u] > dist[farthest]) farthest = u;
27
28             for (int v : adj[u]) {
29                 if (dist[v] == -1) {
30                     dist[v] = dist[u] + 1;
31                     par[v] = u;
32                     q.push(v);
33                 }
34             }
35         }
36         return farthest;
37     }
38
39     // Returns {diameter_length, {endpoint_u, endpoint_v}}
40     pair<int, pair<int, int>> get_diameter() {
41         int u = bfs(1);
42         int v = bfs(u);
43         return {dist[v], {u, v}};
44     }
45
46     // Returns the sequence of vertices on the diameter path from u to
47     // v
48     vector<int> get_path() {
49         auto [len, endpoints] = get_diameter();
50         auto [u, v] = endpoints;
51         vector<int> path;
52         for (int curr = v; curr != -1; curr = par[curr]) {
53             path.pb(curr);
54         }
55         reverse(all(path));
56         return path;
57     };

```

9.6 Euler Tour

9.6.1 EulerTour

```

1 // Euler Tour Technique (Tree Flattening into 1D Array Intervals) - O(V
  // + E)
2 // Subtree of node u corresponds to contiguous range [in_time[u],
  // out_time[u]] in flat_tree
3 struct EulerTour {
4     int n, timer = 0;
5     vector<vector<int>> adj;
6     vector<int> in_time, out_time, flat_tree;
7
8     EulerTour(int n = 0) : n(n), adj(n + 1), in_time(n + 1), out_time(n
  // + 1) {}
9
10    void add_edge(int u, int v) {
11        adj[u].pb(v);
12        adj[v].pb(u);
13    }
14
15    void dfs(int u, int p = -1) {
16        in_time[u] = ++timer;
17        flat_tree.pb(u);
18        for (int v : adj[u]) {
19            if (v != p) dfs(v, u);
20        }
21        out_time[u] = timer;
22    }
23
24    void build(int root = 1) {
25        timer = 0;
26        flat_tree.clear();
27        dfs(root);
28    }
29
30    // Returns [L, R] 1-based index range for subtree queries on node u
31    pair<int, int> get_subtree_range(int u) const {
32        return {in_time[u], out_time[u]};
33    }
34 };

```

10 Game Theory

11 Geometry

11.1 Geometry 2D

11.1.1 Point2D

```

1 // 2D Point / Vector Primitive
2 template <typename T = double>
3 struct Point {
4     typedef Point P;
5     T x, y;
6
7     Point(T x = 0, T y = 0) : x(x), y(y) {}
8
9     bool operator<(P p) const { return tie(x, y) < tie(p.x, p.y); }
10
11    bool operator==(P p) const { return tie(x, y) == tie(p.x, p.y); }
12
13    P operator+(P p) const { return P(x + p.x, y + p.y); }
14
15    P operator-(P p) const { return P(x - p.x, y - p.y); }
16

```

```

17    P operator*(T d) const { return P(x * d, y * d); }
18
19    P operator/(T d) const { return P(x / d, y / d); }
20
21    T dot(P p) const { return x * p.x + y * p.y; }
22
23    T cross(P p) const { return x * p.y - y * p.x; }
24
25    T cross(P a, P b) const { return (a - *this).cross(b - *this); }
26
27    T dist2() const { return x * x + y * y; }
28
29    T dist2(P b) const { return (*this - b).dist2(); }
30
31    double dist() const { return sqrt((double) dist2()); }
32
33    double dist(P b) { return (*this - b).dist(); }
34
35    // angle to x-axis in interval [-pi, pi]
36    double angle() const { return atan2(y, x); }
37
38    P unit() const { return *this / dist(); } // makes dist()=1
39    P perp() const { return P(-y, x); } // rotates +90 degrees
40    P normal() const { return perp().unit(); }
41
42    P getVector(const P &p) const { return p - (*this); }
43
44    // returns point rotated 'a' radians ccw around the origin
45    P rotate(double a) const {
46        return P(x * cos(a) - y * sin(a), x * sin(a) + y * cos(a));
47    }
48
49    friend ostream &operator<<(ostream &os, P p) {
50        // return os << "(" << p.x << ", " << p.y << ")";
51        return os << p.x << " " << p.y;
52    }
53
54    friend istream &operator>>(istream &is, P &p) {
55        return is >> p.x >> p.y;
56    }
57
58    // Project point onto line through a and b (assuming a != b).
59    P projectOnLine(const P &a, const P &b) const {
60        P ab = a.getVector(b);
61        P ac = a.getVector(*this);
62        return a + ab * ac.dot(ab) / a.dist2(b);
63    }
64
65    // Project point c onto line segment through a and b (assuming a !=
66    // b).
67    P projectOnSegment(const P &a, const P &b) const {
68        P ab = b - a, ac = *this - a;
69        long double r = ac.dot(ab), d = a.dist2(b);
70        if (r < 0) return a;
71        if (r > d) return b;
72        return a + ab * r / d;
73    }
74
75    P reflectAroundLine(const P &a, const P &b) const {
76        return projectOnLine(a, b) * 2 - *this;
77    }
78 };
79
80 int dcmp(const ld &a, const ld &b){
81     if(fabs(a - b) < eps)
82         return 0;
83
84     return (a > b ? 1 : -1);
85 }

```

11.1.2 Line2D

```

1 template<class T>
2 double lineDist(T p, T s, T e) {
3     if (s == e) {
4         return s.dist(p);
5     }
6     return fabs((p - s).cross(e - s) / (e - s).dist());
7 }
8
9 template<class T>
10 pair<int, T> lineInter(T s1, T e1, T s2, T e2) {
11     // first = 0 no intersection
12     // first = 1 intersection
13     // first = -1 infinite intersection
14     auto d = (e1 - s1).cross(e2 - s2);
15     if (dcmp(d, 0) == 0) // if parallel same line first = -1 else
16         first = 0
17         return {-(s1.cross(e1, s2) == 0), {0, 0}};
18     auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);
19     return {1, (s1 * p + e1 * q) / d};
20 }
21
22
23 template<class T>
24 bool onSegment(T p, T s, T e) {
25     return dcmp(p.cross(s, e), 0) == 0 && dcmp((s - p).dot(e - p), 0)
26         <= 0;
27 }
28
29 template<class T>
30 double segDist(T p, T s, T e) {
31     if (dcmp((p - s).dot(e - s), 0) <= 0) return s.dist(p);
32     if (dcmp((p - e).dot(e - s), 0) >= 0) return e.dist(p);
33     return lineDist(p, s, e);
34 }
35
36 template<class T>
37 pair<int, T> segInter(T s1, T e1, T s2, T e2) {
38     // first = 0 no intersection
39     // first = 1 intersection

```



```

40 // first = -1 infinite intersection
41 pair<int, T> ret = lineInter(s1, e1, s2, e2);
42 if (ret.first == 0) return ret;
43 else if (ret.first == 1) {
44     if (onSegment(ret.second, s1, e1) && onSegment(ret.second, s2,
45         e2))
46         return ret;
47     else
48         return {0, {0, 0}};
49 } else {
50     if (onSegment(s1, s2, e2) || onSegment(e1, s2, e2))
51         return {-1, (onSegment(s1, s2, e2) ? s1 : e1)};
52     else if (onSegment(s2, s1, e1) || onSegment(e2, s1, e1))
53         return {-1, (onSegment(s2, s1, e1) ? s2 : e2)};
54     else
55         return {0, {0, 0}};
56 }
57
58
59 template<class T>
60 T closestOnSegment(T p, T s, T e) {
61     if ((p - s).dot(e - s) <= 0) return s;
62     else if ((p - e).dot(e - s) >= 0) return e;
63     else return p.projectOnLine(s, e);
64 }
65
66 template<class T>
67 double segSegDist(T s1, T e1, T s2, T e2) {
68     if (segInter(s1, e1, s2, e2).first != 0)
69         return 0;
70     double ret = min({segDist(s1, s2, e2), segDist(e1, s2, e2), segDist(s2,
71         s1, e1), segDist(e2, s1, e1)});
72     return ret;
73 }
74
75
76 template<class T>
77 bool onRay(T p, T s, T e) {
78     return dcmp(p.cross(s, e), 0) == 0 && dcmp((p - s).dot(e - s), 0)
79         >= 0;
80 }
81
82 template<class T>
83 double rayDist(T p, T s, T e) {
84     if ((p - s).dot(e - s) <= 0) {
85         return s.dist(p);
86     }
87     return lineDist(p, s, e);
88 }
89
90 template<class T>
91 pair<int, T> rayInter(T s1, T e1, T s2, T e2) {
92     // first = 0 no intersection
93     // first = 1 intersection
94     // first = -1 infinite intersection
95     pair<int, T> ret = lineInter(s1, e1, s2, e2);
96     if (ret.first == 0) return ret;
97     else if (ret.first == 1) {
98         if (onRay(ret.second, s1, e1) && onRay(ret.second, s2, e2))
99             return ret;
100         else
101             return {0, {0, 0}};
102     } else {
103         if (onRay(s1, s2, e2) || onRay(s2, s1, e1))
104             return {-1, onRay(s1, s2, e2) ? s1 : s2};
105         else
106             return {0, {0, 0}};
107     }
108 }
109
110 template<class T>
111 double rayRayDist(T s1, T e1, T s2, T e2) {
112     if (rayInter(s1, e1, s2, e2).first != 0)
113         return 0;
114     double ret = min(rayDist(s1, s2, e2), rayDist(s2, s1, e1));
115     return ret;
116 }

```

11.1.3 AngleOperations

```

1 // Angle Operations in 2D - [-pi, pi] and [0, 2*pi]
2 template<typename T>
3 ld angleBetween(T a, T b) {
4     ld res = atan2(a.cross(b), a.dot(b));
5     return res < 0 ? res + 2 * PI : res;
6 }
7
8 template<typename T>
9 ld angle0(T a, T o, T b) {
10     return angleBetween(a - o, b - o);
11 }
12
13 ld formatAngle(ld angle) {
14     while (angle > PI) angle -= 2 * PI;
15     while (angle <= -PI) angle += 2 * PI;
16     return angle;
17 }

```

11.1.4 PolygonArea

```

1 // Polygon Area (Shoelace Formula) - O(N)
2 template<typename T>
3 ld polygonArea(const vector<T>& v) {
4     ld area = 0;
5     int n = v.size();
6     for (int i = 0; i < n; i++) {
7         area += v[i].cross(v[(i + 1) % n]);
8     }
9     return 0.5 * abs(area);
10 }

```

11.1.5 Circle3Points

```

1 // Circumcenter & Circumradius of 3 Points
2 typedef Point<double> P;
3
4 bool isColliner(const P &A, const P &B, const P &C) {
5     return dcmp(P(B - A).cross(P(C - A)), 0) == 0;
6 }
7
8 P ccCenter(const P &A, const P &B, const P &C) {
9     P b = C - A, c = B - A;
10    return A + (b * c.dist2() - c * b.dist2()).perp() / b.cross(c) / 2;
11 }
12
13 double ccRadius(const P &A, const P &B, const P &C) {
14     return (B - A).dist() * (C - B).dist() * (A - C).dist() / (2 * abs
15         ((B - A).cross(C - A)));
16 }

```

11.1.6 CircleLineIntersection

```

1 template<class T>
2 vector<T> circleLine(T o, double r, T a, T b) {
3     double h2 = r * r - lineDist(o, a, b) * lineDist(o, a, b);
4     if (dcmp(h2, 0) != -1) { // the line touches the circle
5         T p = o.projectOnLine(a, b); // point P
6         T h = (a - b).unit() * sqrt(h2); // vector parallel to l, of
7             length h
8         if (p - h == p + h)
9             return {p - h};
10        else
11            return {p - h, p + h};
12    }
13    return {};
14 }
15
16 template<class T>
17 vector<T> circleSegment(T o, double r, T a, T b) {
18     vector<T> v = circleLine(o, r, a, b);
19     vector<T> out;
20     for (auto x: v)
21         if (onSegment(x, a, b))
22             out.pb(x);
23     return out;
24 }

```

11.1.7 CircleCircleIntersection

```

1 template<class T>
2 ld circleInter(T a, T b, ld r1, ld r2) {
3     if (r1 > r2) {
4         swap(r1, r2);
5         swap(a, b);
6     }
7     ld d = sqrt((a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y));
8     if (d >= r1 + r2) {
9         return 0;
10    }
11    if (d + r1 <= r2) {
12        return PI * r1 * r1;
13    }
14    ld theta = acos((d * d + r2 * r2 - r1 * r1) / (2 * d * r2));
15    ld alpha = acos((d * d + r1 * r1 - r2 * r2) / (2 * d * r1));
16    ld s1 = theta * r2 * r2;
17    ld t1 = 0.5 * r2 * r2 * sin(2 * theta);
18    ld s2 = alpha * r1 * r1;
19    ld t2 = 0.5 * r1 * r1 * sin(2 * alpha);
20    return (s1 - t1) + (s2 - t2);
21 }

```

11.1.8 ConvexHull

```

1 // Convex Hull (Andrew's Monotone Chain) - O(N log N)
2 // Returns vertices in counter-clockwise order
3 template<typename P>
4 vector<P> convexHull(vector<P> pts) {
5     int n = pts.size(), k = 0;
6     if (n <= 2) return pts;
7     vector<P> h(2 * n);
8     sort(all(pts));
9     for (int i = 0; i < n; i++) {
10        while (k >= 2 && h[k - 2].cross(h[k - 1], pts[i]) <= 0) k--;
11        h[k++] = pts[i];
12    }
13    for (int i = n - 2, t = k + 1; i >= 0; i--) {
14        while (k >= t && h[k - 2].cross(h[k - 1], pts[i]) <= 0) k--;
15        h[k++] = pts[i];
16    }
17    h.resize(k - 1);
18    return h;
19 }

```

11.1.9 RotatingCalipers

```

1 // Rotating Calipers - O(N) Convex Polygon Diameter & Width
2 template<typename T>
3 ll hullDiameter(const vector<T>& S) {
4     int n = S.size(), j = n < 2 ? 0 : 1;
5     ll res = 0;
6     for (int i = 0; i < j; i++) {
7         for (; j = (j + 1) % n; ) {
8             res = max(res, (ll)(S[i] - S[j]).dist2());
9             if ((S[(j + 1) % n] - S[j]).cross(S[i + 1] - S[i]) >= 0)
10                 break;
11         }
12     }
13    return res;
14 }
15
16 template<typename T>
17 ld hullWidth(const vector<T>& S) {
18     int n = S.size();
19     if (n <= 2) return 0;

```

```

19     int j = 1;
20     ld res = 1e18;
21     for (int i = 0; i < n; i++) {
22         while ((S[(i + 1) % n] - S[i]).cross(S[(j + 1) % n] - S[j]) >=
23                 0) j = (j + 1) % n;
24         res = min(res, lineDist(S[j], S[i], S[(i + 1) % n]));
25     }
26     return res;
27 }

```

11.1.10 MinimumEnclosingCircle

```

1 // Minimum Enclosing Circle (Welzl's Algorithm) - O(N) Expected Time
2 typedef Point<double> P;
3
4 P ccCenter(const P& A, const P& B, const P& C) {
5     P b = C - A, c = B - A;
6     return A + (b * c.dist2() - c * b.dist2()).perp() / b.cross(c) / 2;
7 }
8
9 pair<P, double> mec(vector<P> ps) {
10     shuffle(all(ps), mt19937(chrono::steady_clock::now().
11         time_since_epoch().count()));
12     P o = ps[0];
13     double r = 0, EPS = 1 + 1e-9;
14     int n = ps.size();
15     for (int i = 0; i < n; i++) {
16         if ((o - ps[i]).dist() > r * EPS) {
17             o = ps[i]; r = 0;
18             for (int j = 0; j < i; j++) {
19                 if ((o - ps[j]).dist() > r * EPS) {
20                     o = (ps[i] + ps[j]) / 2;
21                     r = (o - ps[i]).dist();
22                     for (int k = 0; k < j; k++) {
23                         if ((o - ps[k]).dist() > r * EPS) {
24                             o = ccCenter(ps[i], ps[j], ps[k]);
25                             r = (o - ps[i]).dist();
26                         }
27                     }
28                 }
29             }
30         }
31     }
32     return {o, r};
33 }

```

11.1.11 HalfPlaneIntersection

```

1 // Basic half-plane struct.
2 template<class T>
3 struct Halfplane {
4
5     // 'p' is a passing point of the line and 'pq' is the direction
6     // vector of the line.
7     T p, pq;
8     long double angle;
9
10     Halfplane() {}
11     Halfplane(const T& a, const T& b) : p(a), pq(b - a) {
12         angle = atan2l(pq.y, pq.x);
13     }
14
15     // Check if point 'r' is outside this half-plane.
16     // Every half-plane allows the region to the LEFT of its line.
17     bool out(const T& r) {
18         return dcmp(pq.cross(r - p), 0) < 0;
19     }
20
21     // Comparator for sorting.
22     bool operator < (const Halfplane& e) const {
23         return angle < e.angle;
24     }
25
26     // Intersection point of the lines of two half-planes. It is
27     // assumed they're never parallel.
28     friend T inter(const Halfplane& s, const Halfplane& t) {
29         long double alpha = (ld)(t.p - s.p).cross(t.pq) / s.pq.cross(t.pq);
30         return s.p + (s.pq * alpha);
31     }
32 };
33
34 template<class T>
35 vector<T> hpIntersect(vector<Halfplane<T>> H) {
36     ld inf = 1e9;
37     T box[4] = { // Bounding box in CCW order
38         Point(inf, inf),
39         Point(-inf, inf),
40         Point(-inf, -inf),
41         Point(inf, -inf)
42     };
43
44     for (int i = 0; i < 4; i++) { // Add bounding box half-planes.
45         Halfplane aux(box[i], box[(i+1) % 4]);
46         H.push_back(aux);
47     }
48
49     // Sort by angle and start algorithm
50     sort(H.begin(), H.end());
51     deque<Halfplane<T>> dq;
52     int len = 0;
53     for (int i = 0; i < int(H.size()); i++) {
54
55         // Remove from the back of the deque while last half-plane is
56         // redundant
57         while (len > 1 && H[i].out(inter(dq[len-1], dq[len-2]))) {
58             dq.pop_back();
59             --len;
60         }
61
62         // Remove from the front of the deque while first half-plane is
63         // redundant
64         while (len > 1 && H[i].out(inter(dq[0], dq[1]))) {
65             dq.pop_front();
66         }
67     }
68 }

```

```

62         --len;
63     }
64
65     // Special case check: Parallel half-planes
66     if (len > 0 && fabs1(H[i].pq.cross(dq[len-1].pq)) < eps) {
67         // Opposite parallel half-planes that ended up checked
68         // against each other.
69         if (dcmp(H[i].pq.dot(dq[len-1].pq), 0) < 0)
70             return vector<T>();
71
72         // Same direction half-plane: keep only the leftmost half-
73         // plane.
74         if (H[i].out(dq[len-1].p)) {
75             dq.pop_back();
76             --len;
77         }
78         else continue;
79     }
80
81     // Add new half-plane
82     dq.push_back(H[i]);
83     ++len;
84
85     // Final cleanup: Check half-planes at the front against the back
86     // and vice-versa
87     while (len > 2 && dq[0].out(inter(dq[len-1], dq[len-2]))) {
88         dq.pop_back();
89         --len;
90     }
91
92     while (len > 2 && dq[len-1].out(inter(dq[0], dq[1]))) {
93         dq.pop_front();
94         --len;
95     }
96
97     // Report empty intersection if necessary
98     if (len < 3) return vector<T>();
99
100    // Reconstruct the convex polygon from the remaining half-planes.
101    vector<T> ret(len);
102    for (int i = 0; i < len; i++) {
103        ret[i] = inter(dq[i], dq[(i+1)%len]);
104    }
105    return ret;
106 }

```

11.2 Geometry 3D

11.2.1 Point3D

```

1 // 3D Point / Vector Primitive
2 template <typename T = double>
3 struct Point3D {
4     typedef Point3D P;
5     typedef const P &R;
6     T x, y, z;
7
8     explicit Point3D(T x = 0, T y = 0, T z = 0) : x(x), y(y), z(z) {}
9
10    bool operator<(R p) const {
11        return tie(x, y, z) < tie(p.x, p.y, p.z);
12    }
13
14    bool operator==(R p) const {
15        return tie(x, y, z) == tie(p.x, p.y, p.z);
16    }
17
18    P operator+(R p) const { return P(x + p.x, y + p.y, z + p.z); }
19
20    P operator-(R p) const { return P(x - p.x, y - p.y, z - p.z); }
21
22    P operator*(T d) const { return P(x * d, y * d, z * d); }
23
24    P operator/(T d) const { return P(x / d, y / d, z / d); }
25
26    T dot(R p) const { return x * p.x + y * p.y + z * p.z; }
27
28    P cross(R p) const {
29        return P(y * p.z - z * p.y, z * p.x - x * p.z, x * p.y - y * p.x);
30    }
31
32    T dist2() const { return x * x + y * y + z * z; }
33
34    double dist() const { return sqrt((double) dist2()); }
35
36    //Azimuthal angle ( longitude) to x=axis in interval [pi, pi]
37    double phi() const { return atan2(y, x); }
38
39    //Zenith angle ( latitude ) to the z=axis in interval [0, pi]
40    double theta() const { return atan2(sqrt(x * x + y * y), z); }
41
42    P unit() const { return *this / (T) dist(); } //makes dist ()=1
43    //returns unit vector normal to *this and p
44    P normal(P p) const { return cross(p).unit(); }
45
46    //returns point rotated 'angle' radians ccw around axis
47    P rotate(double angle, P axis) const {
48        double s = sin(angle), c = cos(angle);
49        P u = axis.unit();
50        return u * dot(u) * (1 - c) + (*this) * c - cross(u) * s;
51    }
52
53    friend ostream &operator<<(ostream &os, P p) {
54        return os << "(" << p.x << ", " << p.y << ")";
55    }
56
57    friend istream &operator>>(istream &is, P &p) {
58        return is >> p.x >> p.y >> p.z;
59    }
60 }

```

```
63 };
64
65
66 namespace Axis {
67     Point3D<double> x(1, 0, 0);
68     Point3D<double> y(0, 1, 0);
69     Point3D<double> z(0, 0, 1);
70 };
```